

Appunti di Ingegneria del Software

Appunti basati sulle lezioni di Davide Falessi

A.A. 2025/2026

Indice

1	Teoria della Misurazione e Scale di Misura	10
1.1	Introduzione alla Misurazione del Software	10
1.1.1	Definizioni Formali	10
1.1.2	Proprietà di una Metrica di Qualità	10
1.2	Tassonomia delle Scale di Misura	11
1.2.1	Scala Nominale e Ordinale	11
1.2.2	Scale Numeriche (Intervallo e Rapporto)	11
1.3	Modelli di Qualità: Lo Standard ISO/IEC 25010	12
1.4	Introduzione alle Metriche Object-Oriented	12
2	GQM+S, Stima dei Costi e Planning Poker	13
2.1	GQM+Strategies (Goal-Question-Metric + Strategies)	13
2.1.1	Il Reference Process (I 6 Step del GQM+S)	13
2.2	Introduzione alla Stima dei Costi	14
2.2.1	Componenti del Costo ed Overheads	14
2.2.2	Il Ruolo del Linguaggio di Programmazione	15
2.3	Tecniche di Stima	15
2.4	Planning Poker	15
2.4.1	Dinamiche e Regole del Gioco	16
2.4.2	Vantaggi e Consigli Pratici (Tips)	16
3	Introduzione all'Architettura Software	17
3.1	Cos'è un'Architettura Software	17
3.1.1	Architetture Intended vs Unintended	17
3.2	Il Modello 4+1 di Kruchten	18
3.3	Progettazione e Decision-Making	18
3.3.1	L'importanza dei Requisiti Non Funzionali	18
3.3.2	Trade-offs e il ruolo dell'Architetto	19
3.4	Documentazione: Viste e Punti di Vista	19
3.5	Software Product Lines (SPL)	19
3.6	Stili Architettureali	20
4	Debito Tecnico e Code Smells	21
4.1	Il Debito Tecnico (Technical Debt)	21
4.1.1	Cause e Conseguenze	21
4.1.2	Strategie di Gestione	22
4.2	Introduzione ai Code Smells	22
4.3	Catalogo dei "Worst Smells"	22
4.3.1	Esempi di Worst Smells	23

5	Machine Learning e Software Analytics per la Software Engineering	24
5.1	Definizione di Machine Learning, ruolo di statistica e informatica	24
5.2	Esempi d'uso: associazioni, classificazione, regressione	24
5.2.1	Apprendimento di associazioni	24
5.2.2	Classificazione	25
5.2.3	Regressione	25
5.3	Tipi di apprendimento: supervised, unsupervised, semi-supervised, reinforcement	25
5.3.1	Supervised learning	25
5.3.2	Unsupervised learning	26
5.3.3	Semi-supervised learning	26
5.3.4	Reinforcement learning	26
5.4	Etica del data mining e del ML	26
5.5	Terminologia: istanze, feature, attributi nominali/numerici, output, modelli	27
5.5.1	Instances	27
5.5.2	Features o attributi	27
5.5.3	Output e modelli	27
5.6	Software quality e costo dei bug	28
5.7	Software analytics: definizione, obiettivi e fonti dati	28
5.8	AI/ML applicati alla Software Engineering	29
5.8.1	Applicazioni alla qualità	29
5.8.2	Applicazioni alla produttività	29
5.9	Defect prediction e ML for software defects	30
5.10	Pipeline: measure, clean data, analyze, model, explain	30
5.11	Estrazione dati da ITS e VCS e raccolta metriche	31
5.12	Identificazione difetti, labeling e defect dataset	31
5.13	Black-box models e necessità di spiegazione	32
5.14	XAI, GDPR, fairness, accountability, transparency	32
5.15	CMMI Level 5 e process improvement	33
5.16	Process chart e controllo statistico	33
5.17	Casi applicativi mostrati nelle slide, anche se solo introdotti	34
5.17.1	Simulatore di release e defect prediction	34
5.17.2	Traceability recovery	34
5.17.3	Defectiveness prediction via JIT	35
5.18	Riepilogo finale	35
6	SZZ: Identifying Buggy Entities	36
6.1	Contesto	36
6.1.1	Cos'è la defect prediction	36
6.1.2	Perché la qualità del dataset è fondamentale	36
6.1.3	Principali fonti di rumore nei dataset	36
6.2	Cos'è SZZ	37
6.2.1	Definizione e obiettivo	37
6.2.2	Uso del versioning e dell'annotation	37
6.2.3	Idea di partire dal fix per risalire al bug-introducing change	37
6.3	Come funziona SZZ	38
6.3.1	Identificazione del fix commit	38
6.3.2	Analisi delle linee modificate	38

6.3.3	Ricerca all'indietro dell'ultima modifica precedente	38
6.3.4	Differenza tra bug-fix change e bug-introducing change	39
6.4	Esempi nelle slide	39
6.4.1	Esempio 1: "When Do Changes Induce Fixes?"	39
6.4.2	Esempio 2: "The Impact of Refactoring Changes on the SZZ Algorithm: An Empirical Study"	40
6.5	Limiti di SZZ	40
6.5.1	Code additions	40
6.5.2	Regressive bugs	41
6.5.3	Refactoring	41
6.5.4	Imprecise backward search	41
6.5.5	Snoring	42
6.6	Da ricordare per l'esame	42
7	Leveraging Defects Lifecycle for Labeling Defective Classes	43
7.1	Contesto e obiettivo del lavoro	43
7.1.1	Defect prediction	43
7.1.2	Importanza del dataset	43
7.1.3	Labeling delle classi difettose	43
7.1.4	Obiettivo generale dello studio	44
7.2	Concetti fondamentali	44
7.2.1	SZZ	44
7.2.2	Earliest AV in JIRA	44
7.2.3	Defect lifecycle	44
7.2.4	Significato di IV, OV, FV e AV	44
7.3	SZZ: funzionamento e limiti	45
7.3.1	Come funziona SZZ	45
7.3.2	Uso di <code>git blame</code> / annotation mechanism	45
7.3.3	Bug introducing commit	45
7.3.4	Limiti del metodo	46
7.4	Aim dello studio e Research Questions	46
7.4.1	Idea di stable life-cycle dei defect	46
7.4.2	Research Questions	46
7.5	RQ1: Is the AV available and consistent?	47
7.5.1	Razionale	47
7.5.2	Design e selezione di progetti e bug	47
7.5.3	Risultati	48
7.5.4	Discussione e implicazioni pratiche	48
7.6	RQ2: Do methods have different accuracy in AV labeling?	48
7.6.1	Razionale	48
7.6.2	Selezione dei progetti	48
7.6.3	Formula della Proportion e significato di P	49
7.6.4	Varianti di Proportion	49
7.6.5	Metodi confrontati	50
7.6.6	Variabili dipendenti e definizione di TP, TN, FP, FN	50

8	Defect Labeling, Evaluation and Metrics	51
8.1	RQ3: Accuratezza nel labeling delle classi	51
8.1.1	Differenza rispetto a RQ2	51
8.1.2	Unità di analisi	51
8.1.3	Design	51
8.1.4	Risultati	51
8.1.5	Confronto tra metodi	52
8.2	RQ4: Identificazione delle feature importanti	52
8.2.1	Razionale	52
8.2.2	Feature selection	52
8.2.3	Correlation analysis	52
8.2.4	Standard Mean Relative Error	52
8.2.5	Risultati	53
8.3	Figure, tabelle e workflow del paper	53
8.3.1	Diagramma del defect lifecycle (Fig. 1.1)	53
8.3.2	Workflow del predicted AV (Fig. 3.1)	53
8.3.3	Workflow del class labeling (Fig. 3.2)	53
8.3.4	Workflow della creazione dei complete datasets (Fig. 3.3)	53
8.3.5	Feature selection e correlation analysis (Fig. 3.4 e Fig. 3.5)	54
8.3.6	Interpretazione di grafici, boxplot e tabelle comparative	54
8.4	Evaluation of Machine Learning Models	54
8.4.1	Training, testing e tuning	54
8.4.2	Perché l'errore sul training set non basta	54
8.4.3	Training set vs test set	54
8.4.4	Dati rappresentativi	54
8.4.5	Problema dei dati limitati	54
8.5	Tecniche di validazione non temporali	55
8.5.1	Holdout	55
8.5.2	Stratification	55
8.5.3	Repeated holdout	55
8.5.4	K-fold cross-validation	55
8.5.5	Stratified k-fold cross-validation e repeated stratified cross-validation	56
8.5.6	Leave-one-out cross-validation	56
8.5.7	Bootstrap	56
8.6	Tecniche di validazione temporali	56
8.6.1	Time-series validation	56
8.6.2	Ordered holdout	56
8.6.3	Walk-forward	56
8.6.4	Importanza dell'ordine temporale	57
8.7	Performance metrics	57
8.7.1	Binary classifier	57
8.7.2	Confusion matrix e metriche base	57
8.7.3	Precision	57
8.7.4	Recall	58
8.7.5	Accuracy	58
8.7.6	Kappa	58
8.7.7	Dipendenza dal contesto applicativo	58
8.8	ROC curves e AUC	58

8.8.1	Threshold dependence	58
8.8.2	ROC curve	58
8.8.3	AUC	59
8.9	Collegamento tra i due PDF	59
8.9.1	Come il labeling influenza la qualità del dataset e del modello . . .	59
8.9.2	Realismo temporale e defect prediction	59
8.9.3	Scegliere bene metriche e validazione	59
8.10	Conclusioni finali	60
8.10.1	Risultati del paper Proportion e limiti dell'approccio Jira-based . .	60
8.10.2	Confronto Proportion vs SZZ	60
8.10.3	Conclusioni su validazione e metriche	60
9	Snoring: rumore nei dataset di defect prediction	61
9.1	Introduzione e contesto della defect prediction	61
9.2	Sleeping defects e snoring classes	61
9.3	Come vengono assegnati i difetti alle classi	62
9.4	SZZ: funzionamento e limiti	62
9.4.1	Funzionamento	62
9.4.2	Limiti	63
9.4.3	Spiegazione del diagramma mostrato nelle slide	63
9.5	Obiettivi dello studio e Research Questions	63
9.6	RQ1: To what extent do defects sleep?	63
9.6.1	Razionale	63
9.6.2	Design	63
9.6.3	Variabili e metriche	64
9.6.4	Risultati	64
9.6.5	Discussione	64
9.7	RQ2: To what extent do classes snore?	64
9.7.1	Razionale	64
9.7.2	Design	64
9.7.3	Variabili e metriche	64
9.7.4	Risultati	65
9.7.5	Discussione	65
9.8	RQ3: To what extent does snoring impact the evaluation of classifiers accuracy?	65
9.8.1	Razionale	65
9.8.2	Design	65
9.8.3	Variabili e metriche	65
9.8.4	Risultati	66
9.8.5	Discussione	66
9.9	RQ4: To what extent does snoring impact defect prediction accuracy? . .	66
9.9.1	Razionale	66
9.9.2	Design	66
9.9.3	Variabili e metriche	66
9.9.4	Risultati	66
9.9.5	Discussione	67
9.10	RQ5: To what extent is no data better than snoring data?	67
9.10.1	Razionale	67

9.10.2	Design	67
9.10.3	Variabili e metriche	67
9.10.4	Risultati	67
9.10.5	Discussione	67
9.11	Conclusioni	68
9.12	Future works	68
10	Effort-Aware Accuracy Metrics e valutazione dei classificatori	69
10.1	Contesto generale	69
10.1.1	Defect prediction e testing activity context	69
10.1.2	Perché l'accuratezza classica non basta	69
10.1.3	Effort-aware accuracy metrics	69
10.2	Concetti fondamentali	70
10.2.1	Definizione di EAM	70
10.2.2	Ranking delle entità difettose	70
10.2.3	Relazione tra effort e LOC da ispezionare	70
10.2.4	Differenza tra ranking per probabilità e ranking effort-aware	70
10.3	PofBx e NPofBx	70
10.3.1	Definizione di PofBx	70
10.3.2	Definizione di NPofBx	70
10.3.3	Significato della normalizzazione	70
10.3.4	Ranking per predicted probability / size	71
10.3.5	Differenza pratica tra i due approcci	71
10.4	Esempio introduttivo	71
10.4.1	Dataset di esempio	71
10.4.2	Significato di PofB20	71
10.4.3	Significato di NPofB20	71
10.4.4	Perché la normalizzazione migliora il numero di difetti trovati	71
10.5	Aim del lavoro e Research Questions	72
10.5.1	Problemi nell'uso delle EAM	72
10.5.2	Proposta di normalizzazione	72
10.5.3	Research Questions	72
10.6	RQ1	72
10.6.1	Numero di studi, progetti, EAM e classifier	72
10.6.2	Risultati principali	72
10.7	RQ2	73
10.7.1	Intuizione di base	73
10.7.2	Variabili indipendente e dipendente	73
10.7.3	PofB10–PofB90 e average	73
10.7.4	Preprocessing	73
10.7.5	Walk-forward	73
10.7.6	Classifier usati	74
10.7.7	Risultati	74
10.7.8	Relative gain	74
10.7.9	Cliff's delta e p-value	74
10.8	RQ3	74
10.8.1	Spearman rank correlation	74
10.8.2	Confronto ranking con e senza normalizzazione	74

10.8.3	Best classifier	75
10.8.4	Risultati e interpretazione pratica	75
10.9	RQ4	75
10.9.1	Confronto tra diverse NPofBx	75
10.9.2	Ruolo di x da 10 a 90	75
10.9.3	Average NPofB e confronto dei ranking	75
10.9.4	Risultati e best classifier	75
10.10	Figure, tabelle e grafici	75
10.10.1	Esempio PofB20 e NPofB20	76
10.10.2	Tabella delle EAM usate negli studi precedenti	76
10.10.3	Tabella dei relative gains	76
10.10.4	Tabella degli equivalent best classifiers	76
10.11	Main results e conclusioni	76
10.11.1	Superiorità delle NPofBs rispetto alle PofBs	76
10.11.2	Cambiamento del best classifier dopo la normalizzazione	76
10.11.3	Implicazioni pratiche	76
10.12	ACUME tool	77
10.12.1	Scopo	77
10.12.2	Input e output	77
10.12.3	Utilità per i ricercatori	77
11	Snoring: rumore nei dataset di defect prediction	78
11.1	Introduzione e contesto della defect prediction	78
11.2	Sleeping defects e snoring classes	78
11.3	Come vengono assegnati i difetti alle classi	79
11.4	SZZ: funzionamento e limiti	79
11.4.1	Funzionamento	79
11.4.2	Limiti	79
11.4.3	Spiegazione del diagramma SZZ nelle slide	80
11.5	Obiettivi dello studio e Research Questions	80
11.6	RQ1: To what extent do defects sleep?	80
11.6.1	Razionale	80
11.6.2	Design	80
11.6.3	Variabili e metriche	80
11.6.4	Risultati	81
11.6.5	Discussione	81
11.7	RQ2: To what extent do classes snore?	81
11.7.1	Razionale	81
11.7.2	Design	81
11.7.3	Variabili e metriche	81
11.7.4	Risultati	81
11.7.5	Discussione	81
11.8	RQ3: To what extent does snoring impact the evaluation of classifiers accuracy?	82
11.8.1	Razionale	82
11.8.2	Design	82
11.8.3	Variabili e metriche	82
11.8.4	Risultati	82

11.8.5	Discussione	82
11.9	RQ4: To what extent does snoring impact defect prediction accuracy? . . .	83
11.9.1	Razionale	83
11.9.2	Design	83
11.9.3	Variabili e metriche	83
11.9.4	Risultati	83
11.9.5	Discussione	83
11.10	RQ5: To what extent is no data better than snoring data?	83
11.10.1	Razionale	83
11.10.2	Design	84
11.10.3	Variabili e metriche	84
11.10.4	Risultati	84
11.10.5	Discussione	84
11.11	Conclusioni	84
11.12	Future works	85
12	Panoramica del modulo di Software Testing	87
12.1	Introduzione al modulo	87
12.2	Informazioni organizzative	87
12.2.1	Canali Teams	87
12.2.2	Ricevimento e comunicazione	88
12.3	Sillabo del modulo	88
12.4	Materiale didattico	88
12.5	Modalità di esame	89
12.6	Attività richieste: generazione dei test	89
12.6.1	Scelta delle classi	89
12.6.2	Test manuali	89
12.6.3	Test automatici	90
12.6.4	Integrazione nel build	90
12.7	Valutazione della qualità dei test	90
12.8	Varianti delle classi e uso degli LLM	91
12.9	Report finale e consegna	91
12.9.1	File <code>classes.txt</code>	92
12.9.2	Invio e discussione orale	92
12.10	Messaggi chiave della lezione	92

Capitolo 1

Teoria della Misurazione e Scale di Misura

1.1 Introduzione alla Misurazione del Software

La misurazione nel software engineering non è un fine, ma uno strumento decisionale di supporto. Come ricordano Lord Kelvin e Tom DeMarco, l'assenza di misurazione impedisce il controllo e, di conseguenza, il miglioramento dei processi e dei prodotti[cite: 330, 341].

1.1.1 Definizioni Formali

Secondo i modelli standard, è necessario distinguere tre concetti fondamentali:

- **Misura (Measure):** Rappresentazione numerica di una caratteristica o di un attributo di un prodotto o processo software (es. 5000 LOC, 30 mesi-uomo)[cite: 363, 364, 365, 368].
- **Metrica (Metric):** La scala o il sistema di misurazione utilizzato per quantificare una misura[cite: 352]. Definisce le regole di assegnazione dei valori (es. Complessità Ciclomatica, Densità dei Difetti)[cite: 353, 354].
- **Misurazione (Measurement):** Il processo pratico di assegnazione dei numeri alle misure utilizzando una metrica[cite: 357].

1.1.2 Proprietà di una Metrica di Qualità

Per essere considerata valida, una metrica deve soddisfare i seguenti criteri:

Rilevanza: Deve misurare un attributo significativo e importante del prodotto o processo software[cite: 380].

Oggettività: Il risultato deve basarsi su dati verificabili e riproducibili, non su opinioni o stime soggettive[cite: 381, 382].

Affidabilità e Consistenza: La metrica deve produrre risultati costanti in condizioni diverse e deve essere coerente con gli obiettivi del progetto[cite: 383, 384].

Usabilità: Il calcolo e l'interpretazione devono essere facili, intuitivi e non richiedere sforzi o risorse eccessive[cite: 385, 386].

Nota del Prof

La pertinenza di una metrica è strettamente legata al contesto di business. Un errore critico in un sistema assicurativo potrebbe riguardare l'accuratezza dei calcoli (oracolo) con conseguenze disastrose, mentre per un motore di ricerca come Google la priorità assoluta risiede nella disponibilità del servizio (uptime), dove anche poche ore di downtime globale risultano inaccettabili.

1.2 Tassonomia delle Scale di Misura

La scelta della scala utilizzata dipende dalla natura dei dati misurati e determina quali operazioni statistiche siano matematicamente applicabili[cite: 407, 459, 477].

1.2.1 Scala Nominale e Ordinale

- **Nominale:** Utilizzata per assegnare valori a etichette o categorie senza un ordine intrinseco o magnitudine (es. tipi di difetti: logica, UI, performance, sicurezza)[cite: 417, 418, 419]. Operazioni ammesse: Moda, frequenza[cite: 459].
- **Ordinale:** Prevede un ordine di rango per le categorie, ma la distanza o magnitudine tra i valori non è definita né significativa (es. priorità dei difetti: Critical, High, Medium, Low)[cite: 422, 423, 424]. Operazioni ammesse: Mediana[cite: 459].

Nota del Prof

Le scale ordinali, pur limitando le analisi statistiche alla mediana, offrono spesso una maggiore oggettività rispetto a scale numeriche forzate. Valutare la severità di un bug su una scala 0-100 introduce un'alta soggettività (es. la differenza tra 65 e 70 è arbitraria), mentre una distinzione qualitativa (Blocker vs Non-Blocker) risulta più robusta, netta e oggettiva.

1.2.2 Scale Numeriche (Intervallo e Rapporto)

In queste scale i valori sono assegnati in ordine di rango e la magnitudine della differenza tra i punti ha un significato reale e misurabile[cite: 427, 433].

- **Intervallo:** La magnitudine delle differenze è significativa, ma non possiede uno zero assoluto reale[cite: 427, 428].
- **Rapporto (Ratio):** È il tipo di scala più sofisticato, in cui la magnitudine delle differenze è calcolabile e possiede uno zero assoluto reale (es. LOC, sforzo, tempo)[cite: 433, 434, 435].

Statistiche ammesse per entrambe: Media, Deviazione Standard, Varianza, Range (per la scala di Rapporto sono ammessi anche coefficiente di variazione e media geometrica)[cite: 459].

1.3 Modelli di Qualità: Lo Standard ISO/IEC 25010

Lo standard ISO 25010 definisce una tassonomia dettagliata degli attributi di qualità (qualità del prodotto e qualità in uso)[cite: 33]. Le caratteristiche principali individuate dallo standard sono:

1. **Functional Suitability (Adeguatezza funzionale):** Completezza, correttezza e adeguatezza funzionale[cite: 39, 40, 41, 42].
2. **Performance Efficiency (Efficienza delle prestazioni):** Comportamento temporale, utilizzo delle risorse e capacità strutturale[cite: 43, 44, 45, 46].
3. **Security (Sicurezza):** Riservatezza, integrità, non ripudio, responsabilità e autenticità[cite: 47, 48, 49, 50, 51, 52].
4. **Maintainability (Manutenibilità):** Modularità, riusabilità, analizzabilità, modificabilità e testabilità[cite: 53, 54, 55, 56, 57, 58].

Nota del Prof

È cruciale distinguere sempre il modello di qualità dalle metriche: il modello (come l'ISO 25010) definisce *cosa* significa la qualità fornendone una tassonomia, mentre le metriche definiscono *come* quantificarla operativamente[cite: 30]. L'adozione di questi standard fallisce sistematicamente se manca la "buona fede" aziendale; l'uso delle metriche al solo scopo di ottenere certificazioni formali di facciata, senza mirare a un miglioramento sostanziale del prodotto, svalisce l'utilità dell'intero sistema di valutazione ingegneristico.

1.4 Introduzione alle Metriche Object-Oriented

Le metriche object-oriented vengono impiegate per misurare la qualità dei sistemi software ad oggetti e identificare tempestivamente difetti di progettazione o futuri problemi di manutenibilità[cite: 68, 69]. Vanno sempre interpretate come indicatori di rischio e valutate in modo comparativo (rispetto a una baseline o a un trend) e non in senso assoluto[cite: 73].

Nota del Prof

In ambito manageriale, è molto più efficace adottare un approccio pratico basato sulle distribuzioni piuttosto che sulle medie globali. Risulta strategicamente più utile concentrare le attività di analisi, refactoring e test mirati esclusivamente sul 10% delle classi che presentano i valori peggiori di complessità (*outliers*), piuttosto che tentare di ottimizzare e riscrivere l'intero sistema in modo uniforme.

Capitolo 2

GQM+S, Stima dei Costi e Planning Poker

2.1 GQM+Strategies (Goal-Question-Metric + Strategies)

L'approccio GQM+Strategies (GQM+S) è una metodologia che combina il metodo classico GQM con l'identificazione delle migliori strategie per raggiungere gli obiettivi. Questo approccio collega a cascata gli obiettivi aziendali alla misurazione:

Business Goals → Software Goals → Questions → Measures → Strategies / Targets.

Nota del Prof

Il CQM originale è stato co-creato dal Professor Cantone. L'estensione "+ Strategies" serve ad associare l'azione tecnica al risultato voluto. Tutto parte dal Business: ogni feature o refactoring viene fatto in primis per raggiungere obiettivi aziendali.

2.1.1 Il Reference Process (I 6 Step del GQM+S)

Il processo si sviluppa in sei step principali per permettere il miglioramento continuo:

0. **Initialize:** Impegno (Commitment), Staffing e Responsabilità.
1. **Characterize:** Definire lo scope dell'applicazione e caratterizzare il contesto ambientale.
2. **Set Goals:** Analisi dello status quo, prioritizzazione degli obiettivi e costruzione della griglia goals-strategies.
3. **Choose Process:** Pianificare l'implementazione delle strategie e organizzare la raccolta/analisi dei dati.
4. **Execute Model:** Esecuzione delle strategie, raccolta dati e feedback.
5. **Analyze Results:** Analisi dei dati, revisione delle strategie e analisi costi/benefici.
6. **Package and Improve:** Adattamento dei goal e registrazione delle "lessons learned".

Nota del Prof

L'ultimo step di "packaging" si basa sul concetto di *Experience Factory*: lo scopo è impacchettare l'esperienza in una knowledge base per riutilizzarla in futuro.

Solo nelle slide (non spiegato a voce)

Esempi di utilizzo tipici:

- **Miglioramento del processo:** Ridurre lead time o rework (Metriche: cycle time, difetti sfuggiti).
- **Project management:** Migliorare prevedibilità (Metriche: milestone burndown, throughput).
- **Supporto decisionale:** Valutare l'adozione di nuove tecnologie (Metriche: esiti progetto pilota).

2.2 Introduzione alla Stima dei Costi

La stima si basa su tre domande fondamentali: quanto sforzo (*effort*) è richiesto, quanto tempo di calendario è necessario e qual è il costo totale dell'attività.

Nota del Prof

Bisogna distinguere l'effort (lavoro puramente impiegato) dal tempo di calendario: un'attività può richiedere poco sforzo ma molto tempo se le risorse non sono subito disponibili.

2.2.1 Componenti del Costo ed Overheads

Il costo include Hardware/Software, viaggi, formazione e l'Effort, che è il fattore dominante (salari e costi sociali). Gli **Overheads** includono:

- Costi degli edifici (riscaldamento, luce).
- Costi di rete e comunicazione.
- Strutture condivise come mensa o biblioteca.

Nota del Prof

Il salario dello sviluppatore a volte non copre neanche il 50% del costo totale per un'azienda a causa degli overhead. Inoltre, il software ha un costo marginale di replicazione quasi nullo: vendere a un secondo cliente non costa nulla di più in termini di produzione.

2.2.2 Il Ruolo del Linguaggio di Programmazione

Solo nelle slide (non spiegato a voce)

Il linguaggio impatta drasticamente su tempi e costi. Ad esempio, l'Assembly richiede molto più tempo per coding (8 settimane per 5000 linee) e testing (10 settimane) rispetto a un linguaggio ad alto livello per lo stesso sistema.

2.3 Tecniche di Stima

IMPORTANTE PER L'ESAME

All'esame non vengono chieste le tecniche a memoria, ma i vantaggi, gli svantaggi e quando usare l'una rispetto all'altra.

- **Algorithmic cost modelling:** Approccio basato su formule e dati storici, generalmente basato sulla dimensione del software (es. COCOMO).
- **Expert judgement:** Uno o più esperti usano la loro esperienza per predire i costi.
 - *Vantaggi:* Metodo relativamente economico; accurato se gli esperti hanno esperienza diretta in sistemi simili.
 - *Svantaggi:* Molto inaccurato se mancano veri esperti nel dominio.
- **Estimation by analogy:** Il costo è calcolato comparando il progetto con uno simile nello stesso dominio.
 - *Vantaggi:* Accurato se sono disponibili dati di progetti passati.
 - *Svantaggi:* Impossibile se non si è mai affrontato un progetto comparabile; richiede un database di costi sistematicamente aggiornato.
- **Parkinson's Law:** Il progetto costa qualunque risorsa sia disponibile. Il lavoro si espande per riempire il tempo disponibile.
 - *Vantaggi:* Non si va mai oltre il budget (*no overspend*).
 - *Svantaggi:* Il sistema rimane solitamente incompleto (*unfinished*).
- **Pricing to win:** Il costo è pari a quanto il cliente può spendere.
 - *Vantaggi:* Permette di vincere il contratto.
 - *Svantaggi:* Bassa probabilità che il cliente ottenga ciò che vuole; i costi non riflettono il lavoro reale richiesto.

2.4 Planning Poker

Il Planning Poker è una tecnica Agile per la stima del consenso, basata sulle carte di James Grenning.

2.4.1 Dinamiche e Regole del Gioco

Ogni partecipante riceve un mazzo di carte con una sequenza numerica, solitamente di **Fibonacci** (1, 2, 3, 5, 8, 13, 21, ecc.).

- **La Sfida:** Il moderatore presenta una *user story* e il team può fare domande al Product Owner.
- **Voto Segreto:** Ogni membro seleziona privatamente una carta che rappresenta la "grandezza" (*size*) del task, considerando tempo, rischio e complessità.
- **Svelamento e Dibattito:** Le carte vengono mostrate insieme. Se non c'è consenso, chi ha dato i valori estremi spiega le proprie ragioni. Il gruppo discute e si vota di nuovo finché non si raggiunge un accordo.

IMPORTANTE PER L'ESAME

Perché i "buchi" di Fibonacci? Servono a forzare la granularità corretta. All'aumentare della grandezza aumenta l'incertezza: la sequenza ci spinge a scegliere tra valori distanti (es. 13 o 21), riflettendo la reale imprecisione della stima per task grandi.

Nota del Prof

In caso di mancato consenso, si sceglie la stima superiore per tutelarsi dalla naturale indole ottimista dell'uomo, che tende a sottostimare i tempi ignorando gli imprevisti.

2.4.2 Vantaggi e Consigli Pratici (Tips)

Il Planning Poker favorisce la collaborazione e fa emergere i problemi tecnici in anticipo attraverso il dibattito.

- **Chi vota:** Solo chi farà materialmente il lavoro. I manager non votano per non influenzare le stime al ribasso.
- **Pareggi:** Se il team è diviso tra due taglie consecutive (es. 5 e 8), si sceglie la maggiore e si procede.

Solo nelle slide (non spiegato a voce)

Suggerimenti per l'efficienza:

- Usare un timer per limitare le discussioni a circa un minuto.
- Includere una carta "I need a break" per segnalare la stanchezza.
- Mantenere una **Baseline** consistente tra le varie iterazioni.

Capitolo 3

Introduzione all'Architettura Software

3.1 Cos'è un'Architettura Software

IMPORTANTE PER L'ESAME

L'architettura software è uno dei pilastri dell'ingegneria del software e rappresenta una delle domande più frequenti in sede d'esame.

Secondo lo standard **IEEE 1471**, l'architettura è definita come:

“L'organizzazione fondamentale di un sistema, incarnata nei suoi componenti, nelle loro relazioni reciproche e con l'ambiente, e i principi che ne governano la progettazione e l'evoluzione.”

L'intento principale, come sostenuto da Kruchten, è fornire un **controllo intellettuale** su sistemi di enorme complessità, permettendo agli stakeholder di comprendere la struttura senza annegare nei dettagli implementativi.

3.1.1 Architetture Intended vs Unintended

Ogni sistema possiede intrinsecamente un'architettura, che può essere classificata in due modi:

- **Intended (Pianificata):** Esplicita e documentata prima della scrittura del codice. Segue l'approccio del *Forward Engineering*.
- **Unintended (Accidentale):** Implicita e spontanea. Emerge quando si scrive codice senza un piano; in questo caso, l'architettura deve essere ricostruita a posteriori (*Reverse Engineering*).

Nota del Prof

Immaginate di costruire una casa: nell'approccio *intended* disegnate prima la pianta. Nell'approccio *unintended* costruite le stanze a casaccio e solo alla fine vi accorgete di che forma ha la casa. Ricordate però il Manifesto Agile: l'architettura ha valore solo se serve a produrre codice migliore, non deve essere burocrazia inutile.

3.2 Il Modello 4+1 di Kruchten

Per gestire la complessità, Kruchten propone di decomporre l'architettura in diverse **viste** basate sul principio della *separation of concerns*.

- **Logical View:** Supporta i requisiti funzionali, ovvero ciò che il sistema deve fare per l'utente finale.
- **Development View:** Si concentra sull'organizzazione dei moduli e dei package nel codice sorgente.

Nota del Prof

Un esempio tipico è lo **stack di Android**, che mostra come il sistema sia stratificato dal Kernel Linux fino alle applicazioni.

- **Dynamic View:** Analizza il comportamento a runtime, i processi e le interazioni tra oggetti. Viene solitamente modellata con i *Sequence Diagram*.

Solo nelle slide (non spiegato a voce)

Completamento del modello:

- **Deployment View:** Si occupa della mappatura fisica del software sui nodi hardware e della distribuzione geografica dei server.
- **Scenarios (il “+1”):** Casi d'uso o sequenze di azioni che fanno da collante tra le altre quattro viste, verificandone la coerenza.

Nota del Prof

Pensate a una casa: potete guardare la pianta elettrica, quella idraulica o quella strutturale. Sono tutte viste diverse dello stesso edificio, necessarie a stakeholder diversi (elettricista, idraulico, ingegnere).

3.3 Progettazione e Decision-Making

Il design architeturale si basa su tre fattori: **Riuso** (di pattern o componenti), **Metodo** (tecniche sistematiche) e **Intuizione** (l'esperienza dell'architetto).

3.3.1 L'importanza dei Requisiti Non Funzionali

Nota del Prof

L'architettura è dettata principalmente dai **Requisiti Non Funzionali** (attributi di qualità come sicurezza, velocità, scalabilità). Questi requisiti sono strutturali: non si può “aggiungere la sicurezza” alla fine del progetto. È come una casa: se volete che sia antisismica, deve nascere così. Non potete renderla antisismica una volta costruita senza abbatterla.

3.3.2 Trade-offs e il ruolo dell'Architetto

L'architetto deve bilanciare forze contrastanti provenienti da stakeholder diversi. Le decisioni sono spesso sub-ottimali e prese “al buio” (quando i requisiti sono ancora vaghi).

Solo nelle slide (non spiegato a voce)

Questo dilemma è rappresentato graficamente dalla scelta tra carne “Cotta” o “Cruda”: ogni scelta comporta un vantaggio per uno stakeholder e uno svantaggio per un altro.

3.4 Documentazione: Viste e Punti di Vista

IMPORTANTE PER L'ESAME

Questa è una distinzione fondamentale. Molti studenti confondono i due termini: ricordateli bene per l'esame.

- **Architectural View (Vista):** È la rappresentazione del sistema da una certa prospettiva. È composta da uno o più *modelli* (o diagrammi).
- **Architectural Viewpoint (Punto di Vista):** È l'insieme di regole, convenzioni e linguaggi usati per costruire la vista.

Nota del Prof

Esempio dello stadio: il *Viewpoint* è il concetto teorico di “guardare dalla tribuna stampa” (regole e angolazione). La *View* è la foto specifica che scattate in quello stadio particolare applicando quel punto di vista. In questo corso, Modello e Diagramma sono considerati sinonimi.

3.5 Software Product Lines (SPL)

Una SPL è una famiglia di prodotti che condividono un set comune di funzionalità (**Core Assets**). Si basa sul *Systematic Reuse*.

- **Scoping:** Decidere cosa includere nella piattaforma comune. Se lo scope è troppo largo si spreca soldi; se è troppo stretto si perde l'opportunità di riuso.
- **Vantaggi:** Riduzione drastica dei tempi e dei costi di sviluppo per i prodotti successivi al primo.

Nota del Prof

Pensate alle auto: la Mercedes usa lo stesso tasto per il finestrino su tutti i modelli. La BMW usa lo stesso motore fisico per vari modelli, limitandone la potenza via software. Nel software il riuso è vitale, ma genera conflitti di budget: il Team A spesso non vuole pagare il costo extra per rendere un modulo riutilizzabile anche dal Team B.

3.6 Stili Architetture

Lo **Stile Architetture** è un pattern astratto; l'**Architettura** è la sua applicazione pratica.

1. **Client/Server:** Il server gestisce dati e sicurezza, i client richiedono servizi. Dissaccoppiamento parziale (il server non conosce i client).
2. **Peer-to-Peer:** Ogni nodo è sia client che server.

Nota del Prof

Come BitTorrent: chi scarica deve anche caricare, evitando colli di bottiglia.

3. **Pipes and Filters:** Stream di dati trasformati in sequenza.

Nota del Prof

Usato in sistemi critici come l'avionica. Performance altissime, riuso quasi nullo.

4. **Shared Data:** Comunicazione tramite un database centrale.

Solo nelle slide (non spiegato a voce)

Si divide in **Repository** (l'agente chiede i dati) e **Blackboard** (il sistema notifica gli agenti quando i dati cambiano).

Capitolo 4

Debito Tecnico e Code Smells

4.1 Il Debito Tecnico (Technical Debt)

Il Debito Tecnico (TD) rappresenta l'applicazione di concetti finanziari al dominio dell'ingegneria del software, aggiungendo criteri di tempo ed economia al classico concetto di manutenibilità.

IMPORTANTE PER L'ESAME

Il debito tecnico si scompone in due concetti finanziari fondamentali che lo distinguono dalla semplice manutenibilità:

- **Capitale (Principal):** Il costo (effort) necessario per eliminare il debito, ovvero il tempo speso per il refactoring.
- **Interessi (Interest):** La penalità futura pagata se il debito non viene eliminato, che si traduce in rallentamento dello sviluppo o maggiore probabilità di bug.

4.1.1 Cause e Conseguenze

Il debito emerge tipicamente per ottimizzazioni a breve termine che creano handicap a lungo termine. Le cause includono la **crescita organica** (aumento naturale della complessità) e **scelte opportunistiche** (trade-off per rispettare le scadenze).

Nota del Prof

Spesso si creano trade-off tra qualità interna e "Time to Market". Seguendo il principio Agile "Working code over comprehensive documentation", gli sviluppatori possono trascurare la modularizzazione o i commenti pur di consegnare software funzionante e farsi pagare .

IMPORTANTE PER L'ESAME

Il debito tecnico impatta sempre la release successiva: se il codice non è manutenibile oggi, lo sviluppo futuro sarà più lento e pronò ai difetti.

Solo nelle slide (non spiegato a voce)

Gli analizzatori statici (come SonarQube) faticano a calcolare gli "Interessi" (il costo futuro), rendendoli attualmente una metrica poco chiara.

4.1.2 Strategie di Gestione

Nota del Prof

Il debito tecnico "scade": se un modulo di scarso valore non verrà mai più modificato, non produrrà interessi. È un errore fare refactoring su codice che non verrà più toccato.

Le strategie di gestione includono:

- **Monitoraggio e Refactoring:** Uso di strumenti come SonarCloud per tracciare il trend delle violazioni.
- **Quality Gate:** Una regola che blocca automaticamente i commit contenenti nuove violazioni.

Nota del Prof

Metafora della barca bucata: la prima cosa da fare non è svuotare l'acqua (vecchio debito), ma tappare il buco (usare il quality gate per non farne entrare di nuovo).

4.2 Introduzione ai Code Smells

Occorre distinguere tra tre concetti:

- **Quality Rule:** Principio ingegneristico validato (es. commenti, bassa complessità).
- **Violation o Code Smell:** Porzione di codice non conforme a una regola di qualità.
- **Defect (Bug):** Problema che impatta la qualità esterna (es. crash).

IMPORTANTE PER L'ESAME

I Code Smells **non sono bug**. Non causano malfunzionamenti immediati ma rendono la manutenzione difficile. Agiscono come un "campo minato", aumentando la probabilità che uno sviluppatore introduca difetti reali.

4.3 Catalogo dei "Worst Smells"

I "Worst Smells" sono violazioni che non portano alcun vantaggio in nessuna circostanza, a differenza di alcuni smell "classici" (come God Class o Code Clones) che possono nascere da trade-off temporali.

4.3.1 Esempi di Worst Smells

- **Modificatori non ordinati:** Crea confusione visiva inutile.
- **Object.finalize() sovrascritto:** In Java, forzare la distruzione manuale è spesso deleterio.
- **Codice commentato:** Sporca la codebase; spesso sono rimasugli di prototipi.
- **Mancanza del "break" in uno Switch:** Causa comportamenti imprevisti proseguendo nei case successivi.
- **Concatenazione di Stringhe con '+' in un ciclo:** Degrada gravemente le performance; il refactoring corretto prevede l'uso di **StringBuilder**.
- **Pessimo Naming:** Esempi includono attributi chiamati A, B o C.

Nota del Prof

Chiamare un metodo `ThisIsAGreatMethod1` non fa risparmiare tempo significativo: non siamo in Formula 1 dove si risparmiano millisecondi.

Solo nelle slide (non spiegato a voce)

L'analisi su 27 progetti Apache ha rivelato che i Worst Smells non sono meno frequenti né più severi degli altri, ma non avendo ragioni di esistere, vanno bloccati sempre tramite Quality Gates.

Capitolo 5

Machine Learning e Software Analytics per la Software Engineering

5.1 Definizione di Machine Learning, ruolo di statistica e informatica

Il **Machine Learning** (ML) è lo studio degli algoritmi che migliorano le proprie performance in un determinato **task** attraverso l'**esperienza**. L'obiettivo è ottimizzare un criterio di performance utilizzando dati di esempio o esperienze passate.

Questa disciplina unisce due campi fondamentali:

- **Statistica**: necessaria per trarre inferenze a partire da un campione di dati.
- **Informatica**: necessaria per fornire algoritmi efficienti in grado di risolvere il problema di ottimizzazione, oltre a rappresentare e valutare il modello per l'inferenza.

Nota del Prof

È fondamentale comprendere che ogni misurazione e ogni modello di ML, sebbene si basi su dati raccolti nel passato, ha come unico scopo quello di **prendere decisioni sul futuro**. L'intero sistema si fonda su un assioma filosofico implicito ma cruciale: assumiamo che il futuro sia simile al passato. Senza questa assunzione di base, l'uso dei dati storici per addestrare un modello sarebbe completamente inutile.

5.2 Esempi d'uso: associazioni, classificazione, regressione

Il Machine Learning viene applicato a diverse tipologie di problemi.

5.2.1 Apprendimento di associazioni

L'**apprendimento di associazioni** calcola la probabilità che un evento si verifichi in concomitanza di un altro. Un esempio tipico è la **basket analysis**, in cui si stima la probabilità che chi compra un certo prodotto acquisti anche un altro prodotto.

5.2.2 Classificazione

La **classificazione** serve a dividere i dati in categorie. Tra gli esempi più importanti troviamo:

- **Credit scoring**: differenziare clienti a basso o alto rischio in base a reddito e risparmi.
- **Face recognition**.
- **Character recognition**.
- **Speech recognition**.
- **Diagnosi medica**.
- **Predizione del click** su pubblicità web.

5.2.3 Regressione

La **regressione** viene utilizzata quando l'output non è una categoria, ma un valore numerico continuo. Un esempio classico è la predizione del **prezzo di un'auto usata** in base ai suoi attributi.

Nota del Prof

Sistemi come Google o Netflix basano il loro intero modello di business sulla predizione. Google non vende solo spazi pubblicitari, ma predice quale utente cliccherà su quale banner per massimizzare i profitti. Netflix usa algoritmi di classificazione e raccomandazione per permettere all'utente di trovare rapidamente il contenuto desiderato. Inoltre, i sistemi di autenticazione biometrica non cercano una corrispondenza pixel per pixel, ma utilizzano il ML per capire quali variazioni sono significative e quali no.

Le principali applicazioni del ML si articolano quindi in tre grandi famiglie: **associazioni**, **classificazione** e **regressione**. Esse coprono problemi di co-occorrenza, categorizzazione e stima di valori numerici.

5.3 Tipi di apprendimento: supervised, unsupervised, semi-supervised, reinforcement

I modelli di ML possono apprendere in modi diversi a seconda dei dati disponibili.

5.3.1 Supervised learning

Nel **supervised learning**, il modello riceve sia i dati di addestramento sia gli output desiderati, cioè le **label**.

Nota del Prof

Un esempio è fornire al modello la dimensione di un tumore dicendogli esplicitamente se in passato si è rivelato benigno o maligno, affinché impari a classificarlo in futuro.

5.3.2 Unsupervised learning

Nell'**unsupervised learning**, vengono forniti dati di addestramento senza output desiderati. Lo scopo è trovare struttura, similarità o gruppi latenti nei dati.

Nota del Prof

In questo caso l'obiettivo non è predire un valore esatto, ma organizzare i dati in cluster simili tra loro, ad esempio per raggruppare geni simili o fare network analysis.

5.3.3 Semi-supervised learning

Solo nelle slide (non spiegato a voce)

Il **semi-supervised learning** utilizza dati di addestramento in cui solo una parte limitata delle istanze possiede le label corrette.

5.3.4 Reinforcement learning

Solo nelle slide (non spiegato a voce)

Il **reinforcement learning** è una tipologia di apprendimento basata su un sistema di ricompense e punizioni. Nelle slide è citato, ma non viene approfondito a voce.

Nel complesso, la distinzione tra questi paradigmi dipende dal tipo di supervisione disponibile e dal modo in cui il sistema riceve informazione utile per apprendere.

5.4 Etica del data mining e del ML

L'utilizzo dei dati solleva questioni etiche rilevanti. L'anonimizzazione dei dati è difficile, e l'uso di certe informazioni può condurre a discriminazioni. Per esempio, utilizzare razza, sesso o religione per l'approvazione di un prestito è eticamente problematico, mentre in ambito medico gli stessi dati possono essere legittimi.

Le domande fondamentali sono:

- Chi ha accesso ai dati?
- Per quale scopo sono stati raccolti?
- Quali conclusioni possono essere tratte legittimamente?

Nota del Prof

Quando un sistema ML prende decisioni, la responsabilità è un tema complesso. Se una Tesla a guida autonoma causa un incidente, oggi la responsabilità legale ricade ancora su chi è al volante, anche in virtù degli accordi di licenza accettati dagli utenti. Inoltre, il ML tende a replicare i bias umani presenti nei dati storici: se nel passato una certa etnia o un certo genere ha subito discriminazioni, il modello può apprendere e riprodurre quel comportamento in futuro.

L'etica del ML impone quindi attenzione a **privacy**, **bias**, **discriminazione** e **responsabilità**. I dati storici non sono neutrali, e i modelli possono ereditare le distorsioni del contesto da cui provengono.

5.5 Terminologia: istanze, feature, attributi nominali/-numerici, output, modelli

Per utilizzare correttamente gli algoritmi di ML è necessario padroneggiare una terminologia standard.

5.5.1 Instances

Le **instances** sono i singoli esempi indipendenti del concetto da apprendere.

Nota del Prof

In un dataset strutturato a tabella, l'istanza rappresenta la singola riga, ad esempio la combinazione di una specifica classe software in una specifica release.

5.5.2 Features o attributi

Le **feature** sono gli aspetti misurabili di un'istanza. Possono essere:

- **Nominali**: valori simbolici o categorici.
- **Numerici**: valori quantitativi su cui si possono eseguire operazioni matematiche.

Nota del Prof

Le feature rappresentano le colonne della tabella. In genere, l'ultima colonna è l'output o variabile da predire, mentre le altre sono metriche che aiutano nella predizione.

5.5.3 Output e modelli

La conoscenza prodotta da un sistema di ML può essere rappresentata tramite:

- **tabelle decisionali**;
- **modelli lineari**;

- alberi decisionali;
- regole;
- cluster.

IMPORTANTE PER L'ESAME

Molti ambienti di ML, in particolare **Weka**, non supportano correttamente gli attributi con scala ordinale, ad esempio *Low*, *Medium*, *High*. Weka tende a trattarli come semplici categorie nominali, perdendo l'informazione dell'ordine. Per questo motivo, spesso occorre eliminare la colonna oppure trasformarla in una variabile binaria, accettando il relativo trade-off nella perdita di informazione.

Un dataset è dunque una tabella in cui le **righe** rappresentano le istanze e le **colonne** rappresentano le feature. La corretta interpretazione del tipo di attributo è essenziale, soprattutto quando si usano strumenti come Weka.

5.6 Software quality e costo dei bug

La **qualità del software** ha un impatto economico enorme. I bug costano a livello globale migliaia di miliardi di dollari, e le attività di **Quality Assurance (QA)** risultano estremamente costose.

Nei sistemi industriali di grandi dimensioni:

- vengono effettuate migliaia di **code review** al giorno;
- vengono eseguiti milioni di **test**;
- ogni modifica deve essere controllata con attenzione.

Nota del Prof

Poiché il testing è molto oneroso, diventa fondamentale stimare in anticipo quali entità del codice, come classi o metodi, abbiano maggiore probabilità di essere difettose, così da concentrare lì l'effort di QA.

Ne segue che la defect prediction nasce anche come risposta economica e organizzativa al problema della qualità: non è sostenibile controllare tutto con la stessa intensità, quindi occorre **prioritizzare**.

5.7 Software analytics: definizione, obiettivi e fonti dati

La **Software Analytics** è la combinazione di **Software Data** e **Data Analytics**. Il software produce una grande quantità di dati distribuiti tra repository, sistemi di build, log di CI/CD, issue tracker, code review e file di configurazione.

Gli obiettivi principali della Software Analytics sono:

1. **Quality improvement**: prevenire crash e bug.

2. **Process improvement:** capire l'impatto di pratiche come code review e rapid release.
3. **Productivity improvement:** valutare l'effetto della CI e di altre pratiche sul team.
4. **Empirical theory building:** costruire teorie empiriche sull'ingegneria del software.

La Software Analytics sfrutta quindi i dati prodotti dagli strumenti di sviluppo per migliorare **qualità**, **processi**, **produttività** e comprensione empirica del software.

5.8 AI/ML applicati alla Software Engineering

L'Intelligenza Artificiale sta trasformando l'Ingegneria del Software.

5.8.1 Applicazioni alla qualità

Tra le applicazioni principali ci sono:

- **defect prediction;**
- **vulnerability prediction;**
- **malware prediction;**
- **test case generation.**

5.8.2 Applicazioni alla produttività

L'AI può supportare anche la produttività del team, ad esempio tramite:

- generazione di **UI**, requisiti, codice e commenti;
- previsione della **produttività** di sviluppatori o team;
- **recommendation** di developer o reviewer;
- identificazione del **turnover** degli sviluppatori.

Solo nelle slide (non spiegato a voce)

Nelle slide vengono citati esempi industriali concreti, come un **AI coding assistant** sviluppato in collaborazione con Mozilla, i tool **SapFix** e **Sapienz** di Facebook per trovare e correggere bug automaticamente, e **Sketch2Code** di Microsoft per trasformare schizzi su lavagna in codice HTML.

Nel contesto della Software Engineering, l'AI non serve quindi solo a prevedere bug, ma anche a supportare gli sviluppatori nella produzione di codice, nel testing e nella gestione del lavoro di squadra.

5.9 Defect prediction e ML for software defects

Nel contesto dei difetti software, il ML ha tre grandi obiettivi:

1. **Predire** i difetti futuri, per ottimizzare le risorse di QA.
2. **Spiegare** perché un software fallisce.
3. **Costruire** teorie empiriche sulla defectiveness.

Nota del Prof

Prevenire è meglio che curare. Eliminare code smells e individuare le classi più rischiose consente di combattere i bug in modo proattivo e di stabilire priorità di testing più efficaci.

La defect prediction non si limita quindi a indicare *dove* è probabile trovare un difetto, ma cerca anche di chiarire *perché* quel rischio esiste e come ridurlo a livello di processo o di codice.

5.10 Pipeline: measure, clean data, analyze, model, explain

L'applicazione della Software Analytics alla defect prediction segue una pipeline precisa:

1. **Measure**: estrazione dei dati grezzi e raccolta delle metriche.
2. **Clean Data**: pulizia dei dati.
3. **Analyze**: ricerca di correlazioni e relazioni tra feature.
4. **Model**: costruzione e addestramento di modelli analitici o black-box.
5. **Explain**: estrazione di conoscenza utile per spiegare le predizioni.

Nota del Prof

Nel corso verrà seguita esattamente questa pipeline: si partirà da una tabella grezza, la si analizzerà, si sceglieranno e si normalizzeranno le colonne, si costruirà il modello e infine si cercherà di estrarre conoscenza utile a evitare in futuro gli stessi errori.

In sintesi

La pipeline consente di trasformare dati grezzi provenienti dai repository in **modelli predittivi** e poi in **conoscenza spiegabile e azionabile**. È uno schema concettuale fondamentale perché collega raccolta dati, modellazione e interpretazione.

5.11 Estrazione dati da ITS e VCS e raccolta metriche

Il primo passo della pipeline consiste nell'estrazione dei dati da due sorgenti principali:

- **ITS (Issue Tracking System)**: ad esempio Jira, che contiene ticket, bug report e informazioni di progetto.
- **VCS (Version Control System)**: ad esempio Git, che contiene codice sorgente, snapshot e log dei commit.

Da queste fonti si raccolgono diverse famiglie di metriche:

- **Code Metrics**: dimensione, complessità, design object-oriented, accoppiamento, coesione.
- **Process Metrics**: numero di commit, churn, difetti pre-release, pratiche di sviluppo.
- **Human Factors**: code ownership, numero di sviluppatori che hanno toccato il file, esperienza dello sviluppatore.

Nota del Prof

Più persone modificano una stessa classe, maggiore è il rischio di introdurre difetti. Inoltre, se un commit è effettuato da uno sviluppatore che non ha esperienza su quella classe, la probabilità di errore cresce sensibilmente.

Combinando dati da **Jira** e **Git**, si ottengono quindi metriche sul codice, sul processo e sui fattori umani, tutte utili per la defect prediction.

5.12 Identificazione difetti, labeling e defect dataset

Per addestrare un modello di defect prediction è necessario costruire un dataset etichettato, in cui ogni classe venga marcata come **difettosa** o **non difettosa**.

Questo processo dipende dal **linkage** tra:

- ticket nell'Issue Tracking System;
- commit nel Version Control System;
- versioni del software coinvolte nel ciclo di vita del difetto.

Nota del Prof

Quando uno sviluppatore effettua un commit per risolvere un problema, inserisce spesso nel messaggio del commit l'ID del ticket Jira associato. Questo ID rappresenta il razionale del cambiamento. Grazie a questa connessione tra commit e ticket, e tramite tecniche come **SZZ** e **Proportion**, è possibile non solo sapere quale commit ha corretto il bug, ma anche risalire a ritroso fino al commit che lo aveva introdotto.

In sintesi

Il **labeling** dipende dalla capacità di collegare correttamente ticket, commit e release. Questo collegamento è alla base della costruzione del **defect dataset** ed è il passaggio che determina la qualità dei dati su cui verranno addestrati i modelli.

5.13 Black-box models e necessità di spiegazione

Molti algoritmi di apprendimento producono modelli **black-box**: ricevono un input e restituiscono una predizione, ma senza rendere trasparente il ragionamento interno che ha portato a quella decisione.

Per esempio, un modello può dire che una certa classe ha alta probabilità di essere difettosa, ma senza spiegare quali fattori abbiano portato a quel risultato.

Nota del Prof

Un predittore come una rete neurale può essere molto accurato, ma non sa giustificare in modo leggibile il perché della sua scelta. Nell'ingegneria del software, invece, capire il motivo della difettosità è cruciale, perché consente al manager di agire sulla causa e non soltanto sul sintomo.

In contesto ingegneristico, quindi, la sola accuratezza non basta: il valore operativo di una predizione aumenta notevolmente quando essa è accompagnata da una spiegazione comprensibile.

5.14 XAI, GDPR, fairness, accountability, transparency

La necessità di spiegare i modelli è legata sia a motivazioni tecniche sia a motivazioni etiche e legali. In questo contesto si colloca il paradigma **FAT**:

- **Fairness**;
- **Accountability**;
- **Transparency**.

L'**Articolo 22 del GDPR** richiede che i processi decisionali automatizzati che hanno impatto sugli individui siano accompagnati da una spiegazione adeguata.

Questo porta a domande fondamentali:

- I sistemi di AI rispettano le leggi?
- I modelli producono decisioni discriminatorie?
- Comprendiamo davvero perché un modello fa una certa predizione?

Nota del Prof

Una tecnica di Explainable AI consiste nell'affiancare a un modello black-box, come una rete neurale, un modello gemello spiegabile, ad esempio un albero decisionale.

Invece di addestrare l'albero direttamente sui dati originali, lo si addestra sugli output generati dalla rete neurale, così da mimarne il comportamento e renderne più leggibili le decisioni.

In sintesi

La **XAI** nasce dall'esigenza di rendere i modelli comprensibili, verificabili e compatibili con vincoli legali ed etici come quelli imposti dal GDPR. In questo quadro, la spiegabilità non è un'aggiunta opzionale, ma una proprietà sempre più centrale.

5.15 CMMI Level 5 e process improvement

Il **CMMI (Capability Maturity Model Integrated)** è uno standard per misurare la maturità dei processi di sviluppo software.

I livelli principali sono:

1. **Initial**
2. **Managed**
3. **Defined**
4. **Quantitatively Managed**
5. **Optimizing**

Il **Level 5**, cioè *Optimizing*, rappresenta il livello più elevato: le performance del processo vengono migliorate in modo continuo tramite innovazione e controllo quantitativo.

Nota del Prof

Il professore riporta un'esperienza diretta in cui ha aiutato un'azienda a mantenere il Livello 5 del CMMI. L'obiettivo era dimostrare statisticamente che certe strategie di miglioramento avessero prodotto benefici reali. I risultati hanno mostrato una forte riduzione dei difetti e un marcato aumento del profitto per dipendente.

Il riferimento al CMMI mostra bene come metriche, controllo statistico e miglioramento continuo possano essere integrati in una visione strutturata della qualità.

5.16 Process chart e controllo statistico

Per controllare la stabilità del processo si utilizzano i **process chart** o **control chart**. Essi servono a:

- identificare problemi mentre si verificano;
- predire l'intervallo atteso dei risultati del processo;
- verificare se il processo è statisticamente stabile;
- decidere se intervenire con correzioni locali o con cambiamenti più strutturali.

La costruzione di un process chart prevede:

1. scelta della variabile da osservare;
2. scelta dell'unità temporale;
3. calcolo della **media**;
4. calcolo dell'**Upper Control Limit** come media più tre deviazioni standard;
5. calcolo del **Lower Control Limit** come media meno tre deviazioni standard.

Nota del Prof

Finché i dati restano all'interno dei limiti di controllo, il processo è considerato statisticamente stabile. Se invece un punto supera uno dei limiti, si attiva una retrospettiva per analizzare la root cause dell'anomalia e capire come evitare che si ripeta.

I **process chart** permettono dunque di monitorare il processo nel tempo e di individuare rapidamente deviazioni anomale che richiedono analisi e intervento.

5.17 Casi applicativi mostrati nelle slide, anche se solo introdotti

Le lezioni presentano anche alcuni casi applicativi concreti.

5.17.1 Simulatore di release e defect prediction

Nota del Prof

È stata sviluppata un'interfaccia per supportare i manager nel **release planning**. Modificando alcuni slider relativi a variabili di input, come la percentuale di God Classes, la compliance architetturale o il numero di nuovi sviluppatori, il modello ricalcolava dinamicamente il numero atteso di difetti nella release successiva. Il sistema forniva anche intervalli di confidenza e scenari di best case e worst case.

5.17.2 Traceability recovery

Solo nelle slide (non spiegato a voce)

Nelle slide si introducono studi sull'uso di tecniche NLP e classificatori di ML per stimare il numero di link rimanenti nel recupero della tracciabilità dei requisiti.

5.17.3 Defectiveness prediction via JIT

Solo nelle slide (non spiegato a voce)

Le slide finali introducono il tema del miglioramento della defect prediction combinando informazioni **Just-In-Time** a livello di commit con metriche classiche a livello di classe e metodo.

Le tecniche discusse trovano applicazione sia in strumenti concreti di supporto decisionale per i manager, sia in linee di ricerca avanzate su **tracciabilità** e **predizione dei difetti**.

5.18 Riepilogo finale

I concetti chiave emersi sono i seguenti:

- Il **Machine Learning** apprende dai dati passati per prendere decisioni sul futuro.
- Le principali famiglie di problemi sono **associazioni**, **classificazione** e **regressione**.
- La **Software Analytics** usa dati provenienti da repository, issue tracker e pipeline di sviluppo per migliorare qualità, processi e produttività.
- La **defect prediction** serve a stimare quali classi o componenti sono più a rischio di contenere difetti.
- La pipeline fondamentale è: **Measure** → **Clean Data** → **Analyze** → **Model** → **Explain**.
- Il **labeling** delle classi dipende dal corretto collegamento tra ticket, commit e release.
- I modelli **black-box** sono potenti ma poco trasparenti, perciò diventa centrale la **Explainable AI**.
- La spiegabilità è importante non solo dal punto di vista tecnico, ma anche per ragioni di **fairness**, **accountability**, **transparency** e conformità al **GDPR**.
- Il **CMMI Level 5** e i **process chart** mostrano come il controllo quantitativo del processo possa tradursi in miglioramenti concreti di qualità e produttività.

Capitolo 6

SZZ: Identifying Buggy Entities

6.1 Contesto

6.1.1 Cos'è la defect prediction

La **defect prediction** è una tecnica che mira a identificare gli **artefatti software** — per esempio classi, metodi o linee di codice — che hanno un'alta probabilità di contenere un difetto. L'obiettivo principale è **ridurre i costi e i tempi del testing e della code review**, permettendo agli sviluppatori di concentrare lo sforzo sulle parti più critiche del sistema.

Nota del Prof

Il testing, sia automatico sia manuale, può essere molto costoso in termini economici e computazionali. Il punto della defect prediction non è “indovinare i bug” in astratto, ma usare i dati storici per **prioritizzare** le attività di qualità. Anche se nel progetto del corso si lavora soprattutto a livello di classe, la stessa logica può essere applicata anche a commit, metodi o linee di codice.

6.1.2 Perché la qualità del dataset è fondamentale

L'**affidabilità di un modello di predizione** dipende direttamente dalla **qualità del dataset** con cui il modello viene addestrato. Se le etichette sono sbagliate o rumorose, il classificatore apprenderà relazioni imprecise e tenderà a produrre risultati poco affidabili.

Nota del Prof

Qui vale in pieno il principio del *Garbage In, Garbage Out*: se i dati in ingresso sono sbagliati, incompleti o rumorosi, anche il modello finale sarà inevitabilmente distorto.

6.1.3 Principali fonti di rumore nei dataset

Tra le principali fonti di rumore citate nelle slide compaiono:

- defect misclassification;
- defect origin.

Solo nelle slide (non spiegato a voce)

Le slide ricordano che lavori precedenti hanno già identificato queste fonti di rumore e hanno proposto tecniche per affrontarle.

Nota del Prof

Un esempio tipico di **misclassification** è quando un ticket classificato come “bug” in realtà descrive una nuova funzionalità o un miglioramento. Un altro problema è il **linkage**: se il commit di fix non contiene l’ID del ticket, allora si perde il collegamento tra issue tracker e version control system, e diventa molto più difficile ricostruire quali classi fossero davvero difettose.

6.2 Cos’è SZZ

6.2.1 Definizione e obiettivo

SZZ è un processo algoritmico usato per identificare **quando un difetto è stato effettivamente introdotto** all’interno di un progetto software. In altre parole, non si limita a dire quale commit ha corretto il problema, ma cerca di risalire al commit precedente che ha inserito la modifica difettosa.

Nota del Prof

Il nome **SZZ** deriva semplicemente dai cognomi degli autori del lavoro originario.

6.2.2 Uso del versioning e dell’annotation

Il metodo sfrutta il **meccanismo di annotation** dei sistemi di versionamento, ad esempio il comando `git blame` in Git. Questo meccanismo permette di associare ogni riga del codice alla revisione che l’ha modificata per ultima.

Nota del Prof

L’annotation è importante perché consente di guardare una riga del file nello stato corrente e chiedersi: *chi l’ha scritta o modificata per ultimo, e in quale commit?* È proprio questa informazione che permette a SZZ di fare la ricerca all’indietro.

6.2.3 Idea di partire dal fix per risalire al bug-introducing change

L’idea centrale di SZZ è partire da un’informazione affidabile nel presente, cioè il **fix commit**, e usarla per ricostruire il passato. In particolare, l’algoritmo osserva **quali linee sono state cambiate per correggere il bug** e poi cerca di capire **quando quelle stesse linee erano state modificate l’ultima volta prima del fix**.

Il commit precedente individuato da questa ricerca viene considerato il **bug-introducing change**, cioè il candidato che ha introdotto il difetto.

IMPORTANTE PER L'ESAME

Il punto chiave da ricordare è questo: **SZZ usa la correzione del bug come indizio per ricostruirne l'origine**. Non parte dal bug report in senso astratto, ma dalla porzione concreta di codice che è stata cambiata per eliminare il problema.

6.3 Come funziona SZZ

6.3.1 Identificazione del fix commit

Il primo passo consiste nell'identificare il **fix commit**, cioè il commit che corregge il difetto.

Nota del Prof

Operativamente, si parte dall'issue tracker, ad esempio Jira, si selezionano i ticket chiusi e classificati come bug, e si cercano i commit collegati a quei ticket. Quel commit rappresenta la **fix** osservabile nel repository.

6.3.2 Analisi delle linee modificate

Una volta trovato il fix commit, SZZ confronta la versione corretta con la versione immediatamente precedente e individua le **linee di codice effettivamente toccate dalla correzione**. Questo passaggio viene normalmente realizzato tramite una **diff**.

L'idea è che il difetto sia collegato proprio a quella parte di codice che è stata modificata nel fix. Perciò SZZ non analizza l'intero file in modo indistinto, ma si concentra sulla porzione di codice che ha subito la correzione.

Nota del Prof

SZZ non “capisce” il significato del bug. Non fa reasoning semantico. Vede solo che certe righe sono state corrette e assume che proprio lì si trovi la traccia storica utile per risalire all'origine del problema.

6.3.3 Ricerca all'indietro dell'ultima modifica precedente

Dopo aver isolato le linee corrette, l'algoritmo esegue una **backward search**. Per ogni riga coinvolta nel fix, usa il meccanismo di annotation per trovare **l'ultima modifica precedente** che aveva scritto o cambiato quella stessa riga.

Quel commit storico viene trattato come il miglior candidato disponibile per il **bug-introducing commit**. L'idea è quindi:

fix osservato nel presente → righe corrette → ricerca storica dell'ultima
modifica precedente

IMPORTANTE PER L'ESAME

SZZ è un algoritmo di **tracciamento storico**, non di comprensione semantica del bug. Funziona bene quando il problema è davvero contenuto nelle linee poi corrette;

funziona peggio quando il bug dipende da contesto, interazioni più ampie o codice mancante.

6.3.4 Differenza tra bug-fix change e bug-introducing change

È fondamentale distinguere chiaramente tra:

- **bug-fix change**: il commit che *risolve* il problema correggendo il codice;
- **bug-introducing change**: il commit passato che, secondo SZZ, ha *introdotto* il difetto, cioè la modifica storica da cui ha origine la porzione di codice poi corretta.

La differenza non è solo temporale, ma anche metodologica: il **bug-fix change** è osservabile con certezza, mentre il **bug-introducing change** è una **stima inferita** tramite backward search.

Nota del Prof

Nel dataset di defect prediction questa distinzione è decisiva. Non basta sapere quale commit ha riparato il bug: bisogna stimare da quale versione in poi una certa classe o un certo file debbano essere considerati difettosi.

6.4 Esempi nelle slide

6.4.1 Esempio 1: “When Do Changes Induce Fixes?”

Solo nelle slide (non spiegato a voce)

Il primo diagramma mostra una sequenza temporale in cui viene segnalato un bug e, in una revisione successiva, viene effettuato un fix. Nel fix commit vengono modificati tre metodi distinti, indicati come `a()`, `b()` e `c()`. SZZ traccia all'indietro la storia di ciascuna di queste porzioni di codice e individua tre revisioni precedenti differenti, ad esempio 1.11, 1.14 e 1.16.

Questo esempio è importante perché mostra che **un singolo bug fix può dipendere da più cambiamenti storici diversi**. Il fix avviene in un unico momento, ma le righe o i metodi corretti possono essere stati introdotti o alterati in revisioni differenti.

Il messaggio didattico è quindi duplice:

1. SZZ lavora a livello di **porzioni di codice**, non a livello astratto di “bug” nel suo complesso;
2. la genealogia del difetto può essere **distribuita** su più commit precedenti.

6.4.2 Esempio 2: “The Impact of Refactoring Changes on the SZZ Algorithm: An Empirical Study”

Nota del Prof

Nel secondo esempio compare un ciclo `for`. Nella versione che introduce il comportamento errato, la condizione è scritta come `for(int i=0; i<=4; i++)`. In seguito al bug report #123, il fix corregge il codice trasformandolo in `for(int i=0; i<4; i++)`.

L'esempio serve a visualizzare il funzionamento operativo di SZZ in modo molto concreto:

1. si individua il **fix commit** grazie al collegamento tra bug report e commit;
2. si esegue la **diff** rispetto alla versione precedente per isolare la riga corretta;
3. si traccia all'indietro la storia di quella riga per capire in quale commit era comparsa la forma difettosa.

Nel caso illustrato, la forma errata è la condizione `i<=4`, che viene poi corretta in `i<4`. SZZ usa proprio questa differenza come indizio per risalire al **bug-introducing change**.

Questo esempio chiarisce bene la logica standard dell'algoritmo: **si osserva il fix, si isolano le righe corrette, si segue la loro storia all'indietro**.

6.5 Limiti di SZZ

Le slide sottolineano che SZZ ha diverse **limitazioni** e che la sua accuratezza non è perfetta. Il problema di fondo è che l'algoritmo si basa su un'assunzione forte: *il bug è stato introdotto nelle stesse linee che vengono poi modificate per correggerlo*. Questa ipotesi è spesso utile, ma non sempre vera.

6.5.1 Code additions

Solo nelle slide (non spiegato a voce)

Le slide riportano che solo il **26–29%** dei **bug-introducing commits** può essere trovato con un algoritmo basato sull'assunzione che un bug sia stato introdotto dalle stesse linee di codice poi modificate per fixarlo.

Questo limite emerge quando il bug non viene corretto modificando una riga sbagliata già esistente, ma **aggiungendo codice che prima mancava**. Per esempio, il fix può consistere nell'inserire un controllo, una condizione o una gestione d'errore assente nella versione difettosa.

In questo caso, SZZ fallisce o diventa molto impreciso, perché non esiste una versione precedente di quella nuova riga da tracciare con `git blame`.

Nota del Prof

Se il difetto dipende da qualcosa che mancava nel codice, l'algoritmo cerca nel passato una riga che in realtà non esisteva ancora. Questo spiega perché i casi di **code addition** siano così problematici per SZZ.

6.5.2 Regressive bugs

I **regressive bugs** sono difetti che emergono dopo modifiche successive del sistema, anche se la riga coinvolta in passato era corretta nel contesto originario.

Il problema è che SZZ tende a tornare alla **prima modifica storica trovata su quella riga**, attribuendo il bug a un commit antico che magari, al momento della sua introduzione, non era affatto errato. La riga può essere diventata problematica solo più tardi, quando il contesto del sistema è cambiato.

Nota del Prof

Qui il limite è concettuale: una riga non è necessariamente “sbagliata in assoluto”. Può diventare sbagliata solo dopo l'introduzione di una nuova feature o dopo una modifica dell'ambiente o dei requisiti. SZZ, però, tende a incolpare chi ha scritto per primo quella riga.

6.5.3 Refactoring

Il **refactoring** cambia la struttura del codice senza modificarne, almeno idealmente, il comportamento funzionale. Esempi tipici sono rinominare variabili, spostare metodi o riorganizzare il codice.

Se la prima modifica precedente trovata da SZZ è proprio un refactoring, l'algoritmo rischia di etichettare quel commit come **bug-introducing**, anche se quel commit non ha realmente introdotto il difetto. In altre parole, il refactoring può **interrompere la pista storica** e nascondere il vero punto di introduzione del bug.

Nota del Prof

SZZ segue la storia delle righe, non il significato profondo della funzionalità. Per questo un refactoring può far sembrare che il bug nasca lì, quando in realtà quella modifica ha solo riscritto o spostato codice senza introdurre il problema.

6.5.4 Imprecise backward search

La **imprecise backward search** indica che SZZ può essere abbastanza efficace nell'individuare **quale porzione di codice** sia collegata al difetto, ma molto meno preciso nello stabilire **da quanto tempo** quella porzione debba essere considerata buggy.

Una riga di codice può essere stata introdotta molto tempo prima in modo corretto e diventare difettosa solo in seguito a cambiamenti dei requisiti, del contesto o delle interazioni con altre parti del sistema. Se SZZ risale troppo indietro, rischia di etichettare come difettose versioni che, storicamente, erano ancora corrette.

IMPORTANTE PER L'ESAME

Questo è uno dei limiti più importanti da ricordare: **SZZ** è spesso più affidabile nel dire “dove guardare” che nel dire con precisione “da quando” il codice sia diventato errato.

6.5.5 Snoring

Solo nelle slide (non spiegato a voce)

Nelle slide lo **snoring** è semplicemente citato nell'elenco delle limitazioni, ma non viene spiegato nel dettaglio.

Nel quadro generale del corso, lo snoring è il fenomeno per cui alcune classi contengono già difetti, ma questi difetti non sono ancora stati scoperti o corretti. Di conseguenza, il dataset può continuare a etichettarle come non difettose.

Il collegamento con SZZ è che l'algoritmo dipende dall'esistenza di un **fix osservabile**. Se un difetto esiste ma non è ancora stato corretto, SZZ non ha alcuna traccia concreta da seguire e quindi non può ricostruirne l'origine.

Nota del Prof

In questa lezione lo snoring viene solo accennato. L'idea essenziale è che SZZ vede soltanto i bug che hanno già lasciato una traccia nella storia dei fix. I bug ancora “dormienti” sfuggono all'algoritmo e contribuiscono a sporcare il labeling del dataset.

6.6 Da ricordare per l'esame

- **Definizione di SZZ:** è un algoritmo che sfrutta il meccanismo di annotation dei sistemi di versionamento per risalire dal **fix commit** al possibile **bug-introducing commit**.
- **Idea chiave:** si parte dalle **linee corrette dal fix** e si cerca l'ultima modifica precedente di quelle stesse linee.
- **Differenza fondamentale:** il **bug-fix change** è il commit che ripara il problema; il **bug-introducing change** è il commit storico che, secondo SZZ, ha introdotto la modifica difettosa.
- **Passaggi essenziali:** identificare il fix commit, eseguire la diff, isolare le righe corrette, usare `git blame` per la backward search.
- **Principali limiti:** **code additions**, **regressive bugs**, **refactoring**, **imprecise backward search** e, in senso più ampio, i problemi legati ai bug non ancora osservabili come nel caso dello **snoring**.

Capitolo 7

Leveraging Defects Lifecycle for Labeling Defective Classes

7.1 Contesto e obiettivo del lavoro

7.1.1 Defect prediction

La **defect prediction** ha l'obiettivo di identificare gli artefatti software, ad esempio classi o file, che hanno un'alta probabilità di manifestare un difetto. Lo scopo principale è ridurre i costi di testing e di code review, permettendo agli sviluppatori di concentrare gli sforzi sulle parti più critiche del codice.

7.1.2 Importanza del dataset

Affinché un modello di predizione sia affidabile, è fondamentale costruire **dataset di difetti di alta qualità**. L'intero lavoro si concentra infatti non tanto sul classificatore in sé, quanto sulla fase precedente, cioè la **costruzione del dataset** e in particolare il **labeling** delle classi come difettose o non difettose.

Se il labeling è inaccurato, il modello apprenderà relazioni sbagliate tra metriche e defectiveness. Per questo motivo, la qualità del dataset non è un dettaglio secondario, ma una condizione necessaria per ottenere risultati credibili.

Nota del Prof

Ci troviamo nella fase iniziale di **misurazione** e creazione del dataset. Prima ancora di addestrare un modello di Machine Learning, bisogna capire come etichettare correttamente le classi che in passato sono state davvero affette da difetti.

7.1.3 Labeling delle classi difettose

Le slide presentano due strategie principali per etichettare le classi come difettose o non difettose:

- usare **SZZ**;
- usare la **Earliest Affected Version (AV)** in **JIRA**, quando disponibile.

Il problema è che entrambe queste fonti possono essere incomplete o inaccurate. Da qui nasce la necessità di studiare metodi automatici alternativi per ricostruire il ciclo di vita del difetto.

7.1.4 Obiettivo generale dello studio

Lo studio ha un duplice obiettivo.

Da un lato, vuole capire **quanto siano disponibili e affidabili le informazioni di AV** inserite dagli sviluppatori nei ticket Jira. Dall'altro, propone un nuovo metodo automatizzato, chiamato **Proportion**, basato sull'idea che i difetti abbiano un **life-cycle relativamente stabile**.

In sintesi, il lavoro vuole:

1. misurare quanto sia realistico affidarsi ai dati di AV presenti in Jira;
2. proporre un metodo alternativo quando l'AV manca o è incoerente;
3. confrontare l'accuratezza del nuovo metodo con quella di diverse varianti di **SZZ**.

7.2 Concetti fondamentali

7.2.1 SZZ

SZZ è l'algoritmo classico per identificare il momento in cui un difetto è stato introdotto nel codice. A partire da un **fix commit**, SZZ usa il meccanismo di annotation del versioning system, per esempio **git blame**, per risalire all'ultima modifica precedente delle linee corrette.

7.2.2 Earliest AV in JIRA

L'approccio **Earliest AV in JIRA** consiste nell'usare la prima **Affected Version** riportata manualmente dagli sviluppatori nel sistema di tracciamento, ad esempio Jira. Se questa informazione fosse sempre presente e consistente, costituirebbe un riferimento molto utile per il labeling.

7.2.3 Defect lifecycle

L'idea centrale del paper è che ogni difetto attraversi un **ciclo di vita** che può essere descritto in termini di versioni del software. Questo ciclo di vita collega il momento in cui il difetto viene introdotto, quello in cui viene osservato e quello in cui viene definitivamente corretto.

7.2.4 Significato di IV, OV, FV e AV

Per descrivere il life-cycle del difetto, il paper usa quattro nozioni fondamentali.

- **IV (Injected Version)**: è la versione in cui il difetto viene introdotto nel codice.

Nota del Prof

È la prima versione in cui il bug esiste davvero nel sistema, anche se magari nessuno se n'è ancora accorto.

- **OV (Opening Version)**: è la versione in cui viene aperto il ticket del bug.

Nota del Prof

È il momento in cui la *failure*, cioè il malfunzionamento osservabile, viene riconosciuta e registrata.

- **FV (Fixed Version)**: è la prima versione rilasciata dopo il *fix commit*.

Nota del Prof

È la prima release in cui quel bug non dovrebbe più comparire.

- **AV (Affected Versions)**: sono tutte le versioni affette dal difetto, cioè l'intervallo di versioni comprese tra il momento di introduzione e quello di correzione.

In termini concettuali, le versioni difettose sono quelle comprese nell'intervallo $[IV, FV)$. Questo significa che la **Fixed Version** non è più affetta dal bug, mentre tutte le versioni precedenti a partire dalla **Injected Version** lo sono.

7.3 SZZ: funzionamento e limiti

7.3.1 Come funziona SZZ

SZZ individua i commit che hanno introdotto il bug tracciando la storia del codice sorgente. L'algoritmo parte da un **fix commit**, cioè da un commit che sappiamo correggere un bug, e usa quella correzione come indizio per cercare all'indietro il possibile **bug-introducing commit**.

7.3.2 Uso di `git blame` / `annotation mechanism`

Operativamente, SZZ prende le linee di codice modificate da un **fix commit** e usa `git blame` per risalire nel tempo, identificando il commit precedente che aveva introdotto o modificato quelle stesse righe.

L'assunzione di fondo è che il bug sia stato introdotto proprio nelle linee poi corrette dal fix. Quando questa assunzione è vera, SZZ può fornire una buona stima del momento di introduzione del difetto.

7.3.3 Bug introducing commit

Il commit individuato tramite backward search viene considerato il **bug-introducing commit**. Da quel momento in poi, la classe o il file coinvolto vengono etichettati come difettosi.

Nota del Prof

In pratica, il commit che ha scritto la forma sbagliata del codice viene trattato come l'origine storica del bug. È un'inferenza plausibile, ma non sempre perfetta.

7.3.4 Limiti del metodo

Le slide chiariscono che SZZ ha limiti strutturali importanti.

- **Non è perfettamente accurato:** il commit trovato da SZZ è una stima, non una verità certa.
- **Non riesce a identificare bene i regression bugs:** un codice può diventare errato solo in seguito a cambiamenti successivi del sistema, anche se in passato era corretto.
- **Non funziona bene quando il fix consiste solo in aggiunta di codice:** se il bug viene corretto introducendo codice nuovo, senza modificare righe già esistenti, SZZ non ha una traccia storica affidabile su cui applicare `git blame`.

Nota del Prof

SZZ può generare sia **falsi positivi** sia **falsi negativi**. Produce falsi positivi quando va troppo indietro nel tempo e attribuisce il difetto a linee che storicamente erano corrette; produce falsi negativi quando non riesce a risalire al vero punto di introduzione del bug.

7.4 Aim dello studio e Research Questions

7.4.1 Idea di stable life-cycle dei defect

L'intuizione centrale dello studio è che i difetti abbiano un **ciclo di vita stabile**. In altre parole, pur variando da bug a bug, sembrerebbe esistere una relazione abbastanza regolare tra:

- il numero di versioni che intercorrono tra introduzione del difetto e sua scoperta;
- il numero di versioni che intercorrono tra scoperta del difetto e sua correzione.

Questa ipotesi motiva il metodo **Proportion**: se tale proporzione è abbastanza stabile, allora è possibile stimare automaticamente la posizione dell'IV anche quando l'AV esplicita manca.

7.4.2 Research Questions

Le domande di ricerca dello studio sono quattro:

- **RQ1:** le **Affected Versions (AV)** sono disponibili e consistenti in Jira?
- **RQ2:** i vari metodi, ad esempio SZZ e Proportion, hanno accuratèzze diverse nell'etichettare le **Affected Versions**?

- **RQ3:** i metodi hanno accuratezze diverse nell'etichettare le **single classi difettose**?
- **RQ4:** l'uso di metodi diversi porta all'identificazione di **feature importanti** diverse nei modelli di predizione?

In questo capitolo ci fermiamo a **RQ1** e **RQ2**.

7.5 RQ1: Is the AV available and consistent?

7.5.1 Rationale

La prima domanda vuole verificare se il campo **Affected Version** sia effettivamente utilizzabile come fonte affidabile per il labeling. Ci sono infatti due problemi distinti:

- **availability:** gli sviluppatori potrebbero non compilare affatto il campo AV;
- **consistency:** il campo potrebbe essere compilato, ma in modo incoerente rispetto all'ordine temporale delle versioni.

Nota del Prof

Per esempio, se la versione più vecchia indicata come affetta risulta successiva all'apertura del ticket, allora quella AV non è coerente e non può essere usata come ground truth affidabile.

7.5.2 Design e selezione di progetti e bug

Per RQ1 sono stati analizzati **212 progetti Apache** collegati sia a **Git** sia a **Jira**. Sono stati considerati solo gli issue report che soddisfacevano simultaneamente i seguenti criteri:

- `Type == Bug`;
- `Status == Closed` oppure `Resolved`;
- `Resolution == Fixed`.

Inoltre, sono stati esclusi:

- i difetti che non avevano un **Git-related fix commit**;
- i difetti che **non erano post-release**.

Dal punto di vista sperimentale:

- la **variabile indipendente** è il singolo **defect**;
- la **variabile dipendente** è la **percentuale di AV disponibili** e di **AV disponibili e consistenti**.

7.5.3 Risultati

I risultati mostrano che il problema è molto rilevante. Sul totale dei difetti selezionati:

- il numero complessivo di bug chiusi e collegati a Git nei 212 progetti Apache è pari a **125.860**;
- **63.539 bug**, cioè circa il **51%**, risultano privi di AV utilizzabile oppure presentano un'AV inaffidabile.

Inoltre, per la maggior parte dei progetti, più del **25%** dei difetti non possiede una AV disponibile e consistente.

7.5.4 Discussione e implicazioni pratiche

La conclusione di RQ1 è netta: l'approccio basato esclusivamente sulla AV inserita in Jira **non è sufficiente** nella maggioranza dei casi.

Se più della metà dei bug non ha una AV affidabile, allora il labeling realistico basato solo sui dati espliciti del ticket diventa spesso impossibile. Questo giustifica la necessità di metodi automatici alternativi.

Nota del Prof

Se non abbiamo un'AV affidabile e non introduciamo un metodo compensativo, finiamo per etichettare come pulite molte classi che invece erano già affette da difetti. Questo produce inevitabilmente molti **falsi negativi** nel dataset.

7.6 RQ2: Do methods have different accuracy in AV labeling?

7.6.1 Razionale

Dato che RQ1 mostra che l'AV è spesso assente o inconsistente, RQ2 si chiede se esistano metodi automatici più accurati per **predire correttamente le versioni affette da un bug**.

La domanda non è più semplicemente "l'AV c'è?", ma: *quando manca, quale metodo ricostruisce meglio l'intervallo delle versioni difettose?*

7.6.2 Selezione dei progetti

Per valutare i metodi in modo affidabile, RQ2 restringe l'analisi a progetti per cui esiste una ground truth più solida. I criteri di selezione sono:

- progetti con più del **50%** di AV disponibili e consistenti;
- progetti con almeno **100 bug** collegati a un Git fix commit e con AV disponibile e consistente;
- progetti con almeno **6 versioni**.

Dopo questa selezione, rimangono **76 progetti Apache Jira**.

7.6.3 Formula della Proportion e significato di P

Il paper introduce una misura proporzionale del ciclo di vita del difetto:

$$P = \frac{FV - IV}{FV - OV}$$

Questa quantità misura, in termini di versioni, il rapporto tra:

- la distanza tra **Fixed Version** e **Injected Version**;
- la distanza tra **Fixed Version** e **Opening Version**.

Una volta stimata P , è possibile ricavare una **Predicted IV** mediante la formula:

$$\text{Predicted IV} = FV - (FV - OV) \cdot P$$

Nota del Prof

L'idea intuitiva è che, pur cambiando la durata assoluta dei singoli bug, la **proporzione** tra momento di scoperta e momento di correzione tenda a essere relativamente stabile. Per questo motivo si prova a usare P come “firma temporale” del lifecycle dei difetti.

7.6.4 Varianti di Proportion

Il paper confronta tre modi diversi di stimare P :

- **Cold Start:** P è calcolato come mediana dei difetti osservati negli altri progetti;
- **Increment:** P è calcolato come media dei difetti corretti nelle versioni precedenti dello stesso progetto;
- **Moving Window:** P è calcolato come media dell'ultimo 1% dei difetti corretti.

Le tre varianti riflettono tre strategie diverse: usare esperienza di altri progetti, usare solo il passato interno del progetto, oppure concentrarsi sul comportamento più recente del processo.

Nota del Prof

A lezione è stata discussa anche una variante chiamata **Total**, che usa tutti i difetti passati e futuri per stimare P . Questa variante non appartiene al paper, ma è utile per capire il problema del realismo temporale.

IMPORTANTE PER L'ESAME

La variante **Total** è concettualmente scorretta in ottica di Machine Learning realistico, perché usa informazioni del futuro per etichettare il passato. In un vero scenario predittivo, il futuro non è disponibile al momento della costruzione del dataset.

7.6.5 Metodi confrontati

I metodi messi a confronto in RQ2 sono:

- **Simple**: assume semplicemente $IV = OV$;
- **SZZ_B** (*Basic*);
- **SZZ_U** (*Unleashed*);
- **SZZ_RA** (*Refactoring Aware*);
- **Proportion_ColdStart**;
- **Proportion_Increment**;
- **Proportion_MovingWindow**;
- le versioni **SZZ_X+**, cioè ciascun metodo SZZ combinato con **Simple**.

7.6.6 Variabili dipendenti e definizione di TP, TN, FP, FN

L'unità di analisi di RQ2 non è ancora la classe-versione, ma la **versione rispetto a un singolo difetto**. In altre parole, si valuta se un metodo etichetta correttamente una certa versione come affetta

Capitolo 8

Defect Labeling, Evaluation and Metrics

8.1 RQ3: Accuratezza nel labeling delle classi

8.1.1 Differenza rispetto a RQ2

Mentre in **RQ2** l'obiettivo era valutare l'accuratezza nell'individuare le **Affected Versions (AV)** di un difetto, in **RQ3** il livello di analisi diventa più fine. Lo scopo non è più soltanto capire se una *versione* sia affetta da un certo bug, ma costruire il dataset più accurato possibile per la **defect prediction a livello di classe**.

In altre parole, RQ2 ragiona ancora in termini di **versione-difetto**, mentre RQ3 passa al problema realmente utile per il dataset finale, cioè stabilire se una **specifica classe** in una **specifica versione** debba essere etichettata come difettosa oppure no.

8.1.2 Rationale

Il rationale di questa ricerca è capire **quale metodo produca il dataset di defect prediction più accurato**. Nel contesto **within-project, across-version, class-level**, la domanda diventa:

una certa classe, in una certa release, deve essere considerata **defective** oppure **non-defective**?

Questa è la forma di labeling che serve davvero per addestrare un classificatore sulle classi software.

8.1.3 Unità di analisi

L'unità di misura non è più la singola versione, ma la coppia **classe-versione**, per esempio *F1.java nella release 1*. Questo significa che ogni istanza del dataset finale rappresenta una classe osservata in una certa release.

8.1.4 Design

Le **variabili indipendenti** rimangono le stesse di RQ2, cioè i diversi metodi di labeling:

- **Simple**;
- **SZZ_B**, **SZZ_U**, **SZZ_RA**;
- **Proportion_ColdStart**, **Proportion_Increment**, **Proportion_MovingWindow**;
- le varianti **SZZ_X+**.

Le **variabili dipendenti** sono le stesse di RQ2, ma cambiano di significato perché l'unità di analisi non è più la versione, bensì la **defectiveness di una classe in una versione**.

Più precisamente:

- **True Positive (TP)**: la classe in una certa versione è realmente difettosa ed è etichettata come difettosa;
- **False Negative (FN)**: la classe in una certa versione è realmente difettosa, ma viene etichettata come non difettosa;
- **True Negative (TN)**: la classe in una certa versione è realmente non difettosa ed è etichettata come non difettosa;
- **False Positive (FP)**: la classe in una certa versione è realmente non difettosa, ma viene etichettata come difettosa.

Da queste quantità vengono poi calcolate le metriche standard:

- **Precision**;
- **Recall**;
- **F1**;
- **MCC**;
- **Kappa**.

8.1.5 Risultati

I risultati mostrano una struttura molto simile a quella già osservata in RQ2, ma con una differenza importante a livello di dominanza dei metodi.

- Come in RQ2, i metodi **Proportion** hanno **Precision** e accuratezza composita, cioè **F1**, **MCC** e **Kappa**, superiori a tutti i metodi **SZZ**.
- Come in RQ2, **SZZ_U** ottiene la **Recall** più alta tra tutti i metodi.
- Come in RQ2, **SZZ_B+** ha la più alta **Precision** all'interno della famiglia **SZZ**.
- Diversamente da RQ2, **Proportion_MovingWindow** emerge come metodo dominante sulle **metriche composite**, risultando il più forte nel labeling delle classi.

8.1.6 Discussione

La discussione di RQ3 porta a tre osservazioni importanti.

La prima è che, confrontando i metodi **SZZ_X** con le corrispondenti varianti **SZZ_X+**, il guadagno nel **class labeling** è **inferiore all'1%**. Quindi, anche qui, l'integrazione con informazioni aggiuntive migliora poco i metodi SZZ sul piano pratico.

La seconda è che **tutti i metodi risultano più accurati nel labeling delle classi che nel labeling delle AV**. Questo è un punto metodologicamente importante: etichettare correttamente una classe è più facile che ricostruire perfettamente l'intero intervallo delle versioni affette da un difetto.

La terza è che questo miglioramento è quantificabile. Rispetto al task di AV labeling, nel task di class labeling si osserva un aumento mediano di:

- **13%** in Precision;
- **5%** in Recall;
- **16%** in F1;
- **27%** in MCC;
- **39%** in Kappa.

Solo nelle slide (non spiegato a voce)

La spiegazione proposta nelle slide è che una singola classe può essere affetta da **più difetti simultaneamente**. Di conseguenza, anche se un metodo manca un difetto specifico, può comunque etichettare correttamente la classe come difettosa grazie alla presenza di un altro difetto concomitante.

Nota del Prof

La Recall di SZZ tende a essere più alta perché SZZ va molto più indietro nel tempo rispetto a Proportion. Questo fa emergere più classi positive, ma al prezzo di moltissimi **falsi positivi**. Quindi SZZ recupera più positivi, ma sporca molto di più il dataset.

8.2 RQ4: Identificazione delle feature importanti

8.2.1 Razionale

RQ4 studia l'effetto a cascata del labeling sulla comprensione del problema. Non basta infatti costruire un dataset etichettato: bisogna anche capire se il metodo di labeling usato influenza **quali feature sembrano importanti** per spiegare la defectiveness.

Il razionale è quindi doppio:

1. capire **quali feature** vengono selezionate come rilevanti dal dataset costruito con un certo metodo;
2. capire se la relazione tra queste feature e la variabile target, cioè la **class defectiveness**, venga preservata correttamente.

Questo è molto importante dal punto di vista pratico, perché le metriche selezionate come più rilevanti guidano anche l'interpretazione ingegneristica del fenomeno: per esempio, si può voler capire se il rischio dipenda da **LOC**, **Churn**, **Age**, **Change Set Size** o altre metriche di processo e prodotto.

8.2.2 Feature selection

Per la **feature selection** si confrontano le feature selezionate usando il dataset prodotto da un certo metodo con le feature selezionate usando il dataset **actual**, cioè quello assunto come riferimento.

Lo scopo è misurare quanto fedelmente un metodo di labeling riesca a preservare la scelta delle feature rilevanti.

Nel materiale del corso viene indicato un approccio di **Exhaustive Search Feature Selection**, implementato tramite Weka e `CfsSubsetEval`. L'idea generale è scegliere feature:

- fortemente informative rispetto alla defectiveness;
- poco ridondanti tra loro.

Per confrontare la correttezza della feature selection vengono riutilizzate metriche analoghe a quelle già viste:

- **TP**;
- **TN**;
- **FP**;
- **FN**;
- **Precision**;
- **Recall**;
- **F1**;
- **MCC**;
- **Kappa**.

8.2.3 Correlation analysis

La seconda metà di RQ4 riguarda la **correlation analysis**. In questo caso non si confrontano più insiemi di feature selezionate, ma il valore della correlazione misurata:

- nel dataset prodotto da un certo metodo;
- nel dataset **actual**.

L'idea è verificare se un metodo di labeling alteri o meno la relazione statistica tra una feature e la defectiveness. Se il labeling è rumoroso, anche la correlazione osservata può risultare distorta.

Nel materiale del corso questa parte viene associata alla **correlazione di Spearman**.

8.2.4 Standard Mean Relative Error

Per quantificare lo scostamento tra la correlazione **predicted** e quella **actual**, viene usato lo **Standard Mean Relative Error**, espresso come:

$$\frac{|\text{Predicted} - \text{Actual}|}{\text{Actual}}$$

Questa formula misura l'errore relativo tra ciò che il metodo stima e ciò che si osserverebbe nel dataset di riferimento.

8.2.5 Risultati

I risultati di RQ4 mostrano in modo molto netto la superiorità dei metodi **Proportion** rispetto ai metodi **SZZ** anche su questo piano.

- I metodi **Proportion** hanno un'accuratezza superiore a tutti i metodi **SZZ** in **tutte e cinque le metriche** considerate.
- I metodi **Proportion** sono gli unici a mostrare una **mediana perfetta** sia in **Precision** sia in **Recall**.
- Diversamente da quanto accade in RQ2 e RQ3, tra le diverse varianti di **SZZ** non emergono grandi differenze.
- Il miglioramento ottenuto passando da **SZZ_X** a **SZZ_X+** resta ancora una volta **inferiore all'1%**.

8.2.6 Discussione

La discussione rende ancora più esplicita la portata dei risultati.

- I metodi **Proportion** hanno un errore che è circa il **25%** dell'errore commesso dai metodi **SZZ**. In altre parole, nel task di identificazione delle feature importanti, **Proportion** sbaglia molto meno.
- A differenza di quanto visto in RQ2 e RQ3, persino il metodo **Simple** risulta più accurato di tutti i metodi **SZZ** in questo task, con un errore che è circa **la metà** di quello di **SZZ**.
- Anche qui il guadagno ottenuto usando le varianti **SZZ_X+** è **praticamente irrilevante**, restando sotto l'1%.

Solo nelle slide (non spiegato a voce)

Le slide sottolineano che la selezione delle feature e l'osservazione delle correlazioni risultano, nel complesso e in media, significativamente più accurate quando si usa il **bugs' life-cycle** rispetto a qualunque metodo **SZZ**.

8.3 Figure, tabelle e workflow del paper

Solo nelle slide (non spiegato a voce)

Le figure, le tabelle e i workflow di questa sezione derivano dalle slide e dal paper, ma non sono stati spiegati a voce in maniera estesa. Conviene quindi leggerli come supporto alla comprensione del flusso metodologico.

8.3.1 Diagramma del defect lifecycle (Fig. 1.1)

Il diagramma del **defect lifecycle** rappresenta la sequenza temporale fondamentale di un bug:

$$IV \rightarrow OV \rightarrow Fix Commit \rightarrow FV$$

dove:

- **IV** è la **Injected Version**;
- **OV** è la **Opening Version**;
- **FV** è la **Fixed Version**.

Le versioni affette dal difetto sono quelle comprese nell'intervallo $[IV, FV)$. Questo significa che il bug esiste già da **IV** e smette di essere presente a partire dalla **FV**.

8.3.2 Workflow del predicted AV (Fig. 3.1)

Il workflow del **predicted AV** mostra come le informazioni provenienti da **Git** e **Jira** vengano combinate.

In particolare:

- da **Git** si ricava il **Bug Fix**;
- da **Jira** si ricavano **Ticket Creation** e **Releases**;
- un certo **Method**, ad esempio **Proportion**, usa queste informazioni per costruire una **Predicted AV**;
- la **Predicted AV** viene confrontata con la **Actual AV** per ricavare **TP**, **TN**, **FP** e **FN**.

8.3.3 Workflow del class labeling (Fig. 3.2)

Il workflow del **class labeling** prende i file o le classi toccate dal **fix** e li combina con l'informazione sulle versioni affette per stabilire, release per release, se una certa classe debba essere considerata difettosa oppure no.

Questo è il passaggio che trasforma un'informazione a livello di **bug/versione** in un'informazione a livello di **classe-versione**, cioè nel formato realmente utile per la defect prediction.

8.3.4 Workflow della creazione dei complete datasets (Fig. 3.3)

Nel workflow dei **complete datasets** al class labeling viene aggiunta la fase di **Metric Collection**. Il risultato finale è una tabella in cui, per ogni istanza classe-versione, si raccolgono:

- l'identificativo della **release**;
- il nome del **file** o della **classe**;
- la label di **defectiveness**;
- varie metriche software, ad esempio **LOC**, **LOC Touched**, **Nfix**, **Churn**, **Age**, **Change Set Size**.

8.3.5 Feature selection e correlation analysis (Fig. 3.4 e Fig. 3.5)

Le figure dedicate a **feature selection** e **correlation analysis** mostrano due flussi paralleli:

- nel primo caso, il dataset predetto e quello actual vengono sottoposti a **feature selection**, e si confrontano le feature selezionate;
- nel secondo caso, sui due dataset si calcolano le correlazioni e si confrontano tramite l'errore relativo.

In entrambi i casi, l'idea è verificare quanto bene il metodo di labeling preservi l'informazione utile per la spiegazione del fenomeno.

8.3.6 Interpretazione di grafici, boxplot e tabelle comparative

I grafici e i boxplot delle slide trasmettono due messaggi centrali.

Il primo è che la variabile P risulta **molto più stabile** rispetto a **IV**, **OV** e **FV**. Le slide osservano infatti che la deviazione standard di P è circa **5** tra progetti diversi e circa **2** all'interno dello stesso progetto. Questo supporta l'ipotesi di **stable life-cycle**.

Il secondo è che, nelle metriche di accuratezza, i metodi **SZZ** tendono a mostrare valori più bassi e meno stabili, soprattutto su **MCC** e **Kappa**, mentre i metodi **Proportion** risultano mediamente più alti e più robusti.

8.4 Evaluation of Machine Learning Models

8.4.1 Training, testing e tuning

La costruzione di un modello di Machine Learning richiede almeno tre momenti distinti:

- **training**, in cui il modello viene addestrato;
- **testing**, in cui il modello viene valutato su dati indipendenti;
- **tuning**, in cui si regolano parametri o scelte modellistiche per migliorarne le prestazioni.

8.4.2 Perché l'errore sul training set non basta

L'errore osservato sul **training set** non è sufficiente per stimare la qualità del modello, perché il modello è stato costruito proprio su quei dati. Una performance alta in training può quindi riflettere semplice adattamento ai dati storici, non reale capacità di generalizzazione.

8.4.3 Training set vs test set

Per valutare seriamente un classificatore, bisogna separare:

- un **training set**, usato per apprendere;
- un **test set**, usato solo per verificare quanto il modello sappia comportarsi su dati non visti.

Nota del Prof

Tutti i dati della tabella sono di tipo *supervised*, quindi conosciamo già l'etichetta corretta. Per simulare il caso reale, si nasconde una parte dei risultati e si usa il modello addestrato sul training set per predirli. Solo dopo si confrontano *actual* e *predicted*.

8.4.4 Dati rappresentativi

Affinché la stima dell'errore abbia senso, sia il training set sia il test set devono essere **rappresentativi** dello stesso problema. Se training e test descrivono popolazioni diverse, la valutazione non misura più correttamente la capacità di generalizzazione.

8.4.5 Problema dei dati limitati

Idealmente, training e test dovrebbero essere grandi. In pratica, però, nei problemi di defect prediction i dati etichettati non sono infiniti, e questo rende necessarie tecniche di validazione che riusino i dati in modo intelligente senza violare il realismo sperimentale.

Nota del Prof

L'assioma implicito di tutta la validazione è che **il futuro sia simile al passato**. Usiamo il passato per stimare come il modello si comporterà su dati futuri che non abbiamo ancora osservato.

8.5 Tecniche di validazione non temporali

Queste tecniche si usano quando l'ordine temporale dei dati non è determinante oppure quando le istanze possono essere trattate come indipendenti.

8.5.1 Holdout

Nel metodo **holdout** il dataset viene diviso una sola volta, per esempio in 2/3 training e 1/3 testing.

Il vantaggio è la semplicità; lo svantaggio è che il campione selezionato per test potrebbe non essere rappresentativo.

Nota del Prof

Se una classe rara finisce tutta nel test set, il modello non l'ha mai vista in training e quindi non può imparare a riconoscerla.

8.5.2 Stratification

La **stratification** è una variante dell'holdout che cerca di mantenere in training e test proporzioni simili della classe di interesse, così da ridurre il rischio di campioni troppo sbilanciati.

8.5.3 Repeated holdout

Nel **repeated holdout** si ripete più volte l'holdout con partizioni diverse e si media l'errore. In questo modo si attenua la dipendenza da una singola divisione sfortunata dei dati.

Il limite è che i test set possono sovrapporsi, e il costo computazionale cresce rapidamente.

Nota del Prof

Se una singola run richiede 5 ore, ripetere l'holdout 5 volte porta facilmente a 25 ore di elaborazione.

8.5.4 K-fold cross-validation

La **k-fold cross-validation** divide il dataset in K parti uguali. A ogni iterazione, una parte viene usata per il test e le altre $K - 1$ per il training. Il processo viene ripetuto K volte, così che ogni istanza finisca nel test set esattamente una volta.

Nota del Prof

L'analogia della pizza è molto efficace: si taglia la pizza in K spicchi, se ne usa uno come test e gli altri come training, poi si cambia spicchio e si ripete finché ogni spicchio è stato usato una volta come test.

8.5.5 Stratified k-fold cross-validation e repeated stratified cross-validation

La versione **stratified** cerca di mantenere la proporzione delle classi in ogni fold. Una pratica comune è il **10 times 10-fold stratified**, che offre stime molto stabili ma richiede molte esecuzioni.

8.5.6 Leave-one-out cross-validation

La **leave-one-out cross-validation** è il caso limite in cui $K = n$, cioè ogni test set contiene una sola istanza. Massimizza l'uso dei dati per il training, ma ha costi computazionali elevati e rende impossibile la stratificazione.

8.5.7 Bootstrap

Il **bootstrap** usa campionamento con reinserimento. Alcune istanze possono comparire molte volte nel training set, mentre altre possono non comparire affatto.

Nota del Prof

Nel contesto del corso questo metodo viene sconsigliato, perché tende a produrre campioni troppo distorti e poco realistici per la defect prediction.

8.6 Tecniche di validazione temporali

Queste tecniche si usano quando l'ordine temporale è parte integrante del problema, come accade nei dataset software basati sull'evoluzione delle release.

8.6.1 Time-series validation

La **time-series validation** preserva l'ordine cronologico dei dati, evitando che il training contenga informazioni provenienti dal futuro rispetto al test.

8.6.2 Ordered holdout

L'**ordered holdout** è un holdout che rispetta la cronologia: la parte iniziale del dataset va nel training set e la parte finale nel test set.

8.6.3 Walk-forward

Nel **walk-forward** il dataset viene diviso in blocchi cronologici, per esempio per release. A ogni passo, si addestra il modello su tutte le release precedenti e si testa su quella successiva.

Nota del Prof

Se l'obiettivo è predire la **Release 4**, allora il training set deve essere composto dall'unione di **Release 1, 2 e 3**. In questo modo il modello usa solo ciò che, realisticamente, era già noto prima della Release 4.

8.6.4 Importanza dell'ordine temporale

Nel contesto della defect prediction, ignorare il tempo significa violare il realismo sperimentale. Un metodo che usa informazioni provenienti dal futuro produce stime artificialmente ottimistiche.

Nota del Prof

Questo è esattamente il problema della variante **Total** discussa a lezione: usare bug futuri per etichettare il passato significa dare al modello un vantaggio che in un vero scenario di predizione non avrebbe mai.

8.7 Performance metrics

8.7.1 Binary classifier

Un **binary classifier** è un modello che assegna ogni istanza a una di due classi, per esempio **Buggy** oppure **Non-Buggy**.

8.7.2 Confusion matrix e metriche base

Le quattro quantità fondamentali della **confusion matrix** sono:

- **TP (True Positive)**: predetto positivo, realmente positivo;
- **FP (False Positive)**: predetto positivo, realmente negativo;
- **TN (True Negative)**: predetto negativo, realmente negativo;
- **FN (False Negative)**: predetto negativo, realmente positivo.

Nota del Prof

Un **False Positive** significa sprecare effort di testing su una classe che in realtà è sana.

Nota del Prof

Un **False Negative** è molto più pericoloso: la classe è davvero difettosa, ma il modello la considera pulita, quindi rischia di arrivare in produzione senza essere testata adeguatamente.

8.7.3 Precision

La **Precision** è definita come:

$$\frac{TP}{TP + FP}$$

e misura quanto siano affidabili le predizioni positive del modello.

Nota del Prof

È la logica di “al lupo al lupo”: se il modello segnala bug ovunque, finirà inevitabilmente con l’avere una Precision bassa.

8.7.4 Recall

La **Recall** è definita come:

$$\frac{TP}{TP + FN}$$

e misura quanti dei positivi reali vengono effettivamente recuperati dal classificatore.

8.7.5 Accuracy

L'**Accuracy** è:

$$\frac{TP + TN}{TP + TN + FP + FN}$$

ma, da sola, può essere fuorviante in presenza di dataset molto sbilanciati.

8.7.6 Kappa

Il **Kappa** misura quanto il classificatore faccia meglio di un classificatore casuale o maggioritario. Vale:

- 1 in caso di classificazione perfetta;
- 0 se la performance equivale al caso;
- valori negativi se il classificatore è peggiore del caso.

Nota del Prof

Nel contesto del corso, un valore di Kappa sufficientemente alto suggerisce che ci sia davvero una relazione empirica non banale tra metriche software e difetti.

8.7.7 Dipendenza dal contesto applicativo

Solo nelle slide (non spiegato a voce)

Le metriche non vanno mai interpretate in isolamento. Per esempio, una Recall molto alta può essere banale da ottenere in dataset fortemente sbilanciati. Bisogna quindi sempre valutare le metriche nel contesto applicativo corretto.

8.8 ROC curves e AUC

8.8.1 Threshold dependence

Metriche come **Precision**, **Recall** e **F1** dipendono dalla soglia usata per trasformare una probabilità in una decisione binaria.

8.8.2 ROC curve

La **ROC curve** rappresenta il compromesso tra:

- **True Positive Rate (TPR)**;
- **False Positive Rate (FPR)**.

Ogni punto della curva corrisponde a una soglia diversa.

8.8.3 AUC

L'**AUC** è l'area sotto la ROC curve e misura, in termini probabilistici, la capacità del modello di assegnare un punteggio più alto a un positivo che a un negativo.

IMPORTANTE PER L'ESAME

Il limite principale dell'AUC, sottolineato a lezione, è che media le prestazioni su **tutte** le soglie possibili, comprese soglie che nel contesto reale potrebbero non avere alcun senso pratico. Per questo motivo, pur essendo utile, non sempre è la metrica più significativa dal punto di vista ingegneristico.

8.9 Collegamento tra i due PDF

Questa parte del materiale mette in relazione due livelli diversi dello stesso problema:

- da un lato il **labeling dei difetti**, cioè la costruzione del dataset;
- dall'altro la **valutazione dei modelli**, cioè il modo in cui giudichiamo la bontà di un classificatore.

8.9.1 Come il labeling influenza la qualità del dataset e del modello

Prima di poter valutare un modello, bisogna costruire un dataset con una ground truth il più possibile affidabile. Se il metodo di labeling è inaccurato, allora il classificatore verrà addestrato su esempi etichettati male e imparerà pattern sbagliati.

Nota del Prof

Questo è un caso classico di *Garbage In, Garbage Out*. Se SZZ sporca il dataset andando troppo indietro nel tempo o etichettando male le classi, il modello imparerà associazioni scorrette e fallirà anche se la fase di validazione è formalmente ben eseguita.

8.9.2 Realismo temporale e defect prediction

L'idea di **realismo temporale** accomuna sia i metodi di labeling sia le tecniche di validazione. Usare solo difetti passati per stimare P , come in **Proportion_Increment**, e usare solo release passate per addestrare il modello, come nel **walk-forward**, riflette lo stesso principio: il modello non deve mai leggere il futuro.

8.9.3 Scegliere bene metriche e validazione

Se lo scopo è supportare decisioni reali su testing e code review, allora servono:

- un **dataset ben etichettato**;
- una **validazione coerente con il tempo**;
- metriche interpretabili, come **F1** e **Kappa**, da leggere insieme e non separatamente.

Un metodo di labeling migliore, come **Proportion**, rende anche più affidabile l'identificazione delle feature importanti, cioè delle vere cause del rischio di difettosità.

8.10 Conclusioni finali

I risultati del paper su **Proportion** e i concetti sulle metriche di valutazione convergono verso una stessa conclusione metodologica: nel Machine Learning applicato alla Software Engineering non basta scegliere un buon classificatore; occorre soprattutto essere rigorosi nella **costruzione del dataset** e nella **validazione del modello**.

8.10.1 Risultati del paper Proportion e limiti dell'approccio Jira-based

Su circa **125.000 difetti**, il lavoro mostra che affidarsi soltanto alle **Affected Versions** presenti in Jira è impossibile in circa il **51%** dei casi, a causa di AV mancanti o incoerenti.

8.10.2 Confronto Proportion vs SZZ

Nei dataset costruiti a livello di classi, i metodi **Proportion** risultano mediamente più accurati di **SZZ** in **Precision**, **F1**, **MCC** e **Kappa**. **SZZ** mantiene una Recall più alta, ma questo dipende dal fatto che tende a etichettare come positive molte più classi, pagando però un prezzo elevato in termini di falsi positivi.

8.10.3 Conclusioni su validazione e metriche

Nessuna metrica da sola è sufficiente. Per giudicare davvero la bontà di un modello di defect prediction bisogna:

- usare un labeling credibile;
- rispettare il tempo con tecniche come **ordered holdout** o **walk-forward**;
- interpretare le metriche in modo congiunto, tenendo conto del contesto applicativo.

Il messaggio finale è quindi che **dataset quality**, **realismo temporale** e **valutazione corretta** sono tre aspetti inseparabili dello stesso problema.

Capitolo 9

Snoring: rumore nei dataset di defect prediction

9.1 Introduzione e contesto della defect prediction

La **defect prediction** è una tecnica che mira a identificare gli artefatti software, come le classi, che hanno un'alta probabilità di presentare un difetto.

- **Obiettivo:** ridurre i costi e il tempo dedicati alle fasi di testing e code review, permettendo agli sviluppatori di concentrare gli sforzi sulle parti più critiche del codice.
- **Contesto:** lo studio si colloca nell'ambito della **within-project defect prediction**, cioè la classificazione delle classi difettose all'interno di una specifica release di un progetto.
- **Problema dei dati:** l'affidabilità di un modello predittivo dipende interamente dalla qualità del dataset su cui viene addestrato.

Nota del Prof

Questo richiama il principio del **Garbage In, Garbage Out**: se i dati in ingresso sono inaccurati, i risultati del classificatore saranno inevitabilmente inaffidabili.

Solo nelle slide (non spiegato a voce)

Lavori precedenti hanno già identificato altre fonti di rumore nei dataset, come la **misclassification** e l'origine dei difetti, proponendo soluzioni mirate.

9.2 Sleeping defects e snoring classes

Gli **sleeping defects** sono difetti che vengono scoperti o corretti solo diverse release dopo la loro introduzione. In altre parole, il difetto resta presente nel codice per un certo intervallo di tempo prima di emergere esplicitamente. Nelle slide questo fenomeno è associato ai **post-release defects**.

Le **snoring classes** sono invece classi affette *esclusivamente* da difetti non ancora corretti. Poiché l'esistenza di un difetto non può essere conosciuta prima del suo fix,

queste classi vengono erroneamente considerate pulite. Di conseguenza, esse introducono **rumore** nei dataset di defect prediction.

Nota del Prof

La metafora è semplice: il difetto *dorme*, ma è la classe che *rusa*, cioè genera disturbo nella misurazione. Se una classe contiene più bug dormienti ma anche un solo bug già noto e corretto, allora viene comunque etichettata come difettosa e non rientra nel fenomeno dello snoring. La classe *rusa* solo quando tutti i bug che la colpiscono sono ancora dormienti.

IMPORTANTE PER L'ESAME

Lo **snoring** introduce nei dataset **solo Falsi Negativi (FN)** e mai Falsi Positivi. Una classe etichettata come difettosa lo è con certezza, perché esiste già un fix che la collega a un difetto. Al contrario, una classe etichettata come non difettosa potrebbe in realtà nascondere un difetto ancora dormiente. Per questo motivo, l'ultima release di un progetto appare sempre artificialmente migliore: i bug possono già esserci, ma non sono ancora stati scoperti.

9.3 Come vengono assegnati i difetti alle classi

Per stabilire da quando una classe è difettosa, il processo segue due strade principali:

1. si considera la prima **Affected Version** riportata nel sistema di tracciamento, ad esempio in Jira;
2. se questa informazione non è disponibile, si utilizza l'algoritmo **SZZ**.

Nota del Prof

Dal punto di vista pratico, si prendono i ticket chiusi e classificati come difetti, si individuano i commit che li correggono e si etichettano come difettose le classi toccate da quei commit, cercando poi di risalire all'indietro fino al momento di introduzione del bug.

9.4 SZZ: funzionamento e limiti

9.4.1 Funzionamento

SZZ è un processo algoritmico che cerca di identificare quando un difetto è stato materialmente introdotto in un progetto software. Il suo funzionamento si basa sui meccanismi di annotazione del sistema di versionamento, come `git blame`.

L'idea è la seguente: a partire dalle righe modificate in un **bug-fix commit**, SZZ cerca di determinare quando quelle stesse righe erano state modificate l'ultima volta prima del fix. In questo modo tenta di risalire al **bug-introducing change**.

9.4.2 Limiti

Un primo limite riguarda le **code additions**. Se il fix di un bug consiste nell'aggiungere codice mancante, SZZ può fallire perché non esiste una precedente modifica della stessa riga da tracciare.

Solo nelle slide (non spiegato a voce)

SZZ riesce a trovare solo il **26–29%** dei **bug-introducing changes**, perché si basa sull'assunzione che il bug sia stato introdotto nelle stesse linee di codice modificate per correggerlo.

9.4.3 Spiegazione del diagramma mostrato nelle slide

Il diagramma riportato nelle slide mostra un caso in cui, nella versione V3, un programmatore corregge un ciclo cambiando una condizione da $i \leq 4$ a $i < 4$ per risolvere il **Bug #123**. Confrontando il fix con la versione precedente tramite una *diff*, SZZ percorre a ritroso la storia di quella riga e individua in **V2** il cambiamento che ha introdotto il difetto.

9.5 Obiettivi dello studio e Research Questions

Lo studio si propone di quantificare l'impatto del rumore introdotto dallo snoring nei dataset di defect prediction. Le domande di ricerca sono cinque:

1. **To what extent do defects sleep?**
2. **To what extent do classes snore?**
3. **To what extent does snoring impact the evaluation of classifiers accuracy?**
4. **To what extent does snoring impact defect prediction accuracy?**
5. **To what extent is no data better than snoring data?**

Le prime due RQ mirano a misurare il fenomeno, mentre le successive valutano il suo impatto sia sulla stima delle prestazioni dei classificatori sia sulla qualità effettiva della previsione.

9.6 RQ1: To what extent do defects sleep?

9.6.1 Rationale

L'obiettivo è comprendere quante release trascorrono tra l'introduzione di un difetto e la sua correzione. In questo modo si misura l'entità del fenomeno dei **difetti dormienti**, che restano invisibili a chi costruisce il dataset.

9.6.2 Design

Sono stati selezionati **20 grandi progetti Apache**, tracciati in **Jira** e versionati con **Git**, con almeno **10 release** e un'elevata qualità nel collegamento tra bug e codice.

9.6.3 Variabili e metriche

- **Variabile indipendente:** nome del bug in Jira.
- **Variabile dipendente:** numero di release per cui il bug ha dormito, misurato come distanza tra il momento di introduzione e quello di fix, sia in valore assoluto sia in percentuale rispetto al numero totale di release.

9.6.4 Risultati

I **boxplot** mostrano che la distribuzione del numero di release dormite è significativamente spostata rispetto allo zero, segnalando che il fenomeno non è marginale ma strutturale.

9.6.5 Discussione

In media, un bug dorme per **7 release**. Inoltre, nella maggior parte dei progetti, un difetto dorme per oltre il **19%** delle release esistenti.

Solo nelle slide (non spiegato a voce)

Esistono anche casi estremi, come il bug **LUCENE-3672**, che ha dormito per **84 release**, e **STRATOS-1653**, rimasto dormiente per **51 release su 53**.

9.7 RQ2: To what extent do classes snore?

9.7.1 Rationale

Un difetto che dorme non produce automaticamente una classe che russa. Se nella stessa classe esiste almeno un altro difetto già noto, la classe viene comunque etichettata come difettosa. Per questo motivo, l'obiettivo non è solo contare i bug dormienti, ma misurare quanto il fenomeno si traduca davvero in **snoring classes**.

9.7.2 Design

L'idea sperimentale consiste nell'osservare un sottoinsieme iniziale del dataset e misurare come cambia la stima della difettosità man mano che il tempo avanza. Nelle slide questo meccanismo è visualizzato tramite i **binocoli**, che rappresentano l'osservatore spostato in avanti nel tempo.

9.7.3 Variabili e metriche

- **Variabile indipendente:** **Progress**, cioè il rapporto tra release osservata e numero totale di release del progetto.
- **Variabile dipendente:** **Missing Rate**, definito come:

$$\frac{FN}{FN + TP}$$

cioè la quota di classi realmente difettose che non vengono riconosciute come tali. Questa quantità corrisponde a **1 - Recall**.

9.7.4 Risultati

Il grafico a linee mostra chiaramente che, all'aumentare del **Progress**, il **Missing Rate** tende a scendere fino ad avvicinarsi allo zero. Ciò significa che serve tempo affinché il dataset riesca a riconoscere correttamente i difetti già presenti.

9.7.5 Discussione

Per la maggior parte dei progetti, il **Missing Rate** resta superiore al **50%** a meno che non si elimini oltre il **20%** delle release più recenti.

Nota del Prof

Per ottenere un **Missing Rate nullo**, in media bisogna ignorare circa il **71%** finale delle release. Questo mostra che la misurazione della difettosità ha bisogno di molto tempo per stabilizzarsi.

9.8 RQ3: To what extent does snoring impact the evaluation of classifiers accuracy?

9.8.1 Rationale

Ricercatori e professionisti confrontano costantemente classificatori, tecniche di feature selection e strategie di tuning. Se però i dataset usati per la valutazione sono contaminati dallo snoring, allora anche le conclusioni sperimentali risultano **biasate**.

9.8.2 Design

Sono stati impiegati **15 classificatori** differenti. La validazione è stata effettuata con una strategia di **ordered holdout**, mantenendo l'ordine cronologico dei dati: **66%** per il training e **33%** per il testing.

9.8.3 Variabili e metriche

Le variabili dipendenti considerate sono:

- **TP, TN, FP, FN;**
- **Precision;**
- **Recall;**
- **F1;**
- **Cohen's Kappa;**
- **AUC;**
- **Matthews Correlation Coefficient.**

Inoltre viene calcolato il **Relative Bias**, cioè la distanza relativa tra l'accuratezza di un classificatore misurata su dati rumorosi e quella misurata su dati privi di rumore.

La **variabile indipendente** è la presenza o assenza dello **snoring** nei set di training e di testing.

9.8.4 Risultati

I boxplot mostrano un **Relative Bias** molto elevato per quasi tutte le metriche. I valori medi riportati sono particolarmente alti per **Recall**, **Kappa** e **Matthews**, mentre risultano sensibilmente più bassi per **AUC**.

9.8.5 Discussione

Lo snoring può portare a premiare il classificatore sbagliato.

Solo nelle slide (non spiegato a voce)

Quando la valutazione viene eseguita su dati affetti da snoring, il classificatore realmente migliore viene selezionato correttamente solo nel **45%** dei casi.

9.9 RQ4: To what extent does snoring impact defect prediction accuracy?

9.9.1 Rationale

Dopo aver mostrato che lo snoring altera la valutazione dei classificatori, occorre misurare quanto esso peggiori anche la loro **accuratezza predittiva reale**.

9.9.2 Design

L'accuratezza viene testata addestrando i modelli su dati rumorosi, indicati come **TrS**, e valutandoli su un test set privo di snoring, indicato come **TeNS**. In questo modo il deterioramento osservato dipende dalla qualità del training set e non da un rumore condiviso anche nel test.

9.9.3 Variabili e metriche

Le variabili e le metriche sono le stesse della RQ3.

9.9.4 Risultati

Il confronto tra i boxplot mostra un peggioramento sistematico delle prestazioni. La degradazione interessa tutti i classificatori e tutte le metriche osservate.

9.9.5 Discussione

Lo snoring degrada pesantemente le performance di tutti i **15 classificatori**. La metrica meno colpita risulta essere la **Precision**.

IMPORTANTE PER L'ESAME

La **Recall** crolla mentre la **Precision** si riduce molto meno perché lo snoring introduce soprattutto **Falsi Negativi**. Se il modello viene addestrato con dati che fanno apparire molte classi difettose come non difettose, tenderà a essere molto conservativo e a prevedere quasi sempre la classe pulita. In questo modo mancherà moltissimi veri difetti, facendo precipitare la **Recall**; tuttavia, le poche volte in cui segnalerà un difetto, tenderà a farlo su casi relativamente più solidi, mantenendo la **Precision** più stabile.

9.10 RQ5: To what extent is no data better than snoring data?

9.10.1 Rationale

Poiché non è possibile eliminare con precisione solo i singoli datapoint rumorosi, una possibile strategia consiste nel rimuovere intere release recenti, che sono le più esposte al rischio di contenere difetti ancora dormienti. La domanda è quindi se sia preferibile avere **meno dati ma più puliti**.

9.10.2 Design

L'esperimento confronta diversi scenari in cui vengono rimosse le ultime **1, 2, 3 o 4 release** dal training set. Le classi già esplicitamente etichettate come difettose vengono mantenute, perché non possono rappresentare falsi negativi.

9.10.3 Variabili e metriche

- **Variabile indipendente:** numero di release eliminate.
- **Variabile dipendente:** guadagno relativo sulle metriche di accuratezza.

9.10.4 Risultati

Rimuovere l'ultima release produce in media un miglioramento significativo, in particolare su **Recall** e **F1**. Al contrario, rimuovere tre o quattro release porta a un peggioramento delle prestazioni.

9.10.5 Discussione

Ignorare una sola release è generalmente meglio che usare l'intero dataset. Tuttavia, eliminare troppe release impoverisce eccessivamente il training set.

Nota del Prof

Qui emerge il conflitto tra due principi ingegneristici. Da un lato c'è il **Garbage In, Garbage Out**: se si mantengono dati freschi ma rumorosi, il modello viene contaminato. Dall'altro c'è il principio per cui **la dimensione del dataset conta**: se si eliminano troppe release, il modello perde informazione utile e peggiora. Il compromesso migliore, cioè lo *sweet spot*, consiste nello scartare soltanto le ultime **1 o 2 release**.

9.11 Conclusioni

Dallo studio emerge che il fenomeno dei **sleeping defects** e delle **snoring classes** ha un impatto molto forte sulla costruzione dei dataset di defect prediction.

- la maggior parte dei difetti dorme per oltre il **19%** del ciclo di vita del progetto;
- il **Missing Rate** resta superiore al **50%** finché non si rinuncia ad analizzare almeno il **20%** finale delle release;
- l'errore relativo nella valutazione dei classificatori si aggira intorno al **100%** per quasi tutte le metriche, con l'eccezione dell'**AUC**;
- lo snoring penalizza in particolare **Recall**, **F1**, **Matthews** e **Cohen's Kappa**;
- rimuovere l'ultima release può migliorare sensibilmente la qualità della prediction.

9.12 Future works

Le possibili estensioni del lavoro includono:

- lo sviluppo di tecniche di rimozione del rumore più **fine-grained**;
- la combinazione di più tecniche di mitigazione del rumore;
- l'uso di distribuzioni probabilistiche come input dei modelli predittivi, invece di etichette binarie rigide;
- la replica dello studio nel contesto della classificazione **Just-In-Time (JIT)**.

Capitolo 10

Effort-Aware Accuracy Metrics e valutazione dei classificatori

10.1 Contesto generale

Nel campo dell'**ingegneria del software**, i modelli di **defect prediction** sono classificatori di Machine Learning con lo scopo di identificare gli artefatti software, come classi o metodi, che hanno un'alta probabilità di manifestare un difetto.

10.1.1 Defect prediction e testing activity context

Il fine ultimo della **defect prediction** è ridurre il costo delle attività di testing, analisi o *code review*. Prevedendo dove si annidano i bug, il team di sviluppo può dare priorità agli sforzi, concentrandoli su porzioni specifiche di codice, invece di ispezionare l'intero sistema alla cieca.

10.1.2 Perché l'accuratezza classica non basta

Storicamente, i modelli predittivi venivano valutati tramite metriche classiche derivate dal mondo dell'**Information Retrieval**, come **Precision** o **Recall**.

Nota del Prof

Il professore sottolinea che, sebbene le metriche classiche siano utili a livello teorico, risultano inadeguate e poco pratiche nel contesto del testing. Sapere che un classificatore ha una Precision dello 0.90 o una Recall dello 0.89 non fornisce al manager alcuna indicazione pratica sull'impatto reale in termini di *effort* necessario per ispezionare il software. Per questo motivo, le metriche classiche risultano troppo astratte.

10.1.3 Effort-aware accuracy metrics

Per colmare questo divario, sono nate le **Effort-Aware Accuracy Metrics (EAM)**. Si tratta di metriche di accuratezza che tengono esplicitamente in considerazione l'**effort**, cioè il costo o il tempo richiesto per ispezionare il codice suggerito dal classificatore, garantendo così una stima più realistica dei benefici apportati.

10.2 Concetti fondamentali

10.2.1 Definizione di EAM

Le **Effort-Aware Metrics** misurano i benefici forniti da un classificatore in base alla sua capacità di ordinare correttamente le entità candidate a essere difettose, bilanciando i bug trovati con lo sforzo speso per cercarli.

10.2.2 Ranking delle entità difettose

Poiché è impossibile analizzare tutto il software, file e classi vengono inseriti in un **ranking**, cioè una lista ordinata. Il tester ispezionerà il codice partendo dalla cima della lista e scendendo verso il basso finché non esaurisce le risorse a disposizione.

10.2.3 Relazione tra effort e LOC da ispezionare

Nota del Prof

Nell'approccio EAM si assume, come approssimazione, che l'effort necessario per testare un'entità sia linearmente proporzionale alla sua dimensione fisica, cioè al numero di **Linee di Codice (LOC)**. Ispezionare una classe enorme costerà quindi molto più effort rispetto a una classe molto piccola.

10.2.4 Differenza tra ranking per probabilità e ranking effort-aware

L'approccio classico ordina semplicemente le classi in base alla **probabilità** di contenere un bug. Un approccio realmente **effort-aware**, invece, ordina le classi tenendo conto anche di quanto *costa* ispezionarle in termini di LOC.

10.3 PofBx e NPofBx

10.3.1 Definizione di PofBx

La **PofBx** è la EAM non normalizzata più nota. Definisce la percentuale di bug che un tester può identificare ispezionando il top $x\%$ delle linee di codice del sistema, seguendo un ranking basato esclusivamente sulla **probabilità predetta** di difetto.

10.3.2 Definizione di NPofBx

La **NPofBx** è la metrica proposta dagli autori del paper. Rappresenta la versione **normalizzata** della PofBx. Mantiene la stessa logica generale, ma utilizza un ranking differente.

10.3.3 Significato della normalizzazione

Invece di ordinare semplicemente per probabilità predetta, la NPofBx usa un ranking basato sulla **predicted defect density**, cioè una densità predetta dei difetti.

10.3.4 Ranking per predicted probability / size

La densità viene calcolata dividendo la probabilità predetta di un'entità per la sua dimensione:

$$\frac{\text{predicted probability}}{\text{size}}$$

10.3.5 Differenza pratica tra i due approcci

Nota del Prof

Senza normalizzazione, il modello potrebbe suggerire di testare per prima una classe enorme con probabilità del 90% di contenere un bug, esaurendo subito il budget di LOC analizzabili. Con la normalizzazione, il modello capisce che è più conveniente testare per prime diverse classi piccole ma promettenti, trovando molti più bug a parità di LOC lette.

10.4 Esempio introduttivo

Per chiarire l'effetto della normalizzazione, le slide e l'audio presentano un esempio pratico.

10.4.1 Dataset di esempio

Si considera un sistema di **1790 LOC**, diviso in **7 entità**, di cui **3** effettivamente difettose.

10.4.2 Significato di PofB20

Si assume di poter utilizzare il 20% del budget di ispezione. Il 20% di 1790 LOC corrisponde a **358 LOC**. Ordinando le 7 entità per semplice **probabilità predetta**, le prime due entità consumano tutto il budget disponibile. Analizzandole, si trova solo **1** delle **3** entità difettose. Di conseguenza:

$$PofB20 = 33\%$$

10.4.3 Significato di NPofB20

Applicando la normalizzazione, cioè ordinando le entità per Probabilità/Size, le classi piccole e promettenti risalgono la graduatoria. Con lo stesso tetto di 358 LOC, si riescono ad analizzare **3** entità anziché 2, trovando **2** delle **3** entità difettose. Di conseguenza:

$$NPofB20 = 66\%$$

10.4.4 Perché la normalizzazione migliora il numero di difetti trovati

Semplicemente cambiando il criterio di ordinamento del ranking, senza alterare i dati originali né il classificatore, il numero di bug trovati **raddoppia**.

10.5 Aim del lavoro e Research Questions

10.5.1 Problemi nell'uso delle EAM

Il lavoro nasce dall'osservazione che esiste una grave carenza, e talvolta un uso scorretto, delle EAM in letteratura, il che ne mina la validità e la generalizzazione.

10.5.2 Proposta di normalizzazione

L'obiettivo del lavoro è duplice:

1. evidenziare le problematiche storiche nell'uso delle EAM;
2. proporre e valutare la validità della metrica normalizzata **NPofB**, basata sulla **predicted defect density**.

10.5.3 Research Questions

Le quattro domande di ricerca sono:

- **RQ1**: quali EAM vengono usate nei paper di Ingegneria del Software?
- **RQ2**: la normalizzazione migliora effettivamente i valori di PofBs?
- **RQ3**: il ranking dei classificatori cambia se si normalizza la PofB?
- **RQ4**: il ranking dei classificatori cambia al variare della x nelle NPofBx normalizzate?

10.6 RQ1

Per rispondere alla prima domanda, gli autori hanno effettuato uno **Systematic Mapping Study**.

10.6.1 Numero di studi, progetti, EAM e classifier

Sono stati analizzati **152 primary studies** provenienti dai principali journal del settore. Lo studio finale è stato validato su **14 progetti** — 12 open-source e 2 industriali — confrontando **10** diverse soglie di EAM e usando **10 classificatori**.

10.6.2 Risultati principali

I risultati mostrano che:

- la grande maggioranza della letteratura ignora le EAM: **132 studi su 152** usano **0 EAM**, valutando i modelli solo tramite Precision o Recall;
- i pochi studi che usano EAM si limitano quasi sempre a **una sola metrica**, tipicamente **PofB20**, riducendo la generalizzabilità dei risultati;
- sette studi su venti presentavano un **mismatch nei nomi delle metriche**, sintomo di scarsa comprensione o assenza di standardizzazione.

10.7 RQ2

10.7.1 Intuizione di base

La RQ2 indaga se la **NPofB** ottenga performance matematicamente migliori rispetto alla **PofB**.

10.7.2 Variabili indipendente e dipendente

Nota del Prof

Il professore richiama una nozione cruciale di ingegneria degli esperimenti. In questo studio, la **variabile indipendente** è la presenza o assenza della normalizzazione, mentre la **variabile dipendente** è lo score della PofB calcolato con o senza normalizzazione.

10.7.3 PofB10–PofB90 e average

La valutazione è stata eseguita su un intero spettro di soglie, dal **10%** al **90%** a passi di 10, escludendo 0 e 100 perché banali, più una metrica media, **AveragePofB**. In totale vengono considerate **10 PofBs**.

10.7.4 Preprocessing

Prima dell'esperimento, i dati sono stati preprocessati tramite:

1. **Log10 normalization**;
2. **Feature subset selection**;
3. **SMOTE**.

Nota del Prof

La trasformazione in $\log_{10}$ è fondamentale. Se si usano feature disomogenee, per esempio secondi, numero di bug o LOC, passare al logaritmo base 10 rende le distribuzioni più omogenee e aumenta l'accuratezza di base.

10.7.5 Walk-forward

Nota del Prof

Il **walk-forward** è vitale nell'ingegneria del software. Si dividono i dati in release sequenziali: per predire la Release 4 si usa come training set l'unione delle release 1, 2 e 3. Questo impedisce al modello di leggere il futuro e preserva il realismo temporale.

10.7.6 Classifier usati

Solo nelle slide (non spiegato a voce)

Sono stati usati 10 algoritmi ML ben noti: **Decision Table**, **IBk (k-NN)**, **J48**, **KStar**, **Naive Bayes**, **SMO**, **Random Forest**, **Logistic Regression**, **BayesNet** e **Bagging**.

10.7.7 Risultati

La normalizzazione incrementa i valori della PofB in modo statisticamente significativo e di svariati ordini di grandezza.

10.7.8 Relative gain

Il guadagno relativo viene calcolato come:

$$\frac{NPofB - PofB}{PofB}$$

Il **relative gain** cresce fortemente per soglie più basse: per esempio, il guadagno della **PofB10** è circa doppio rispetto a quello della **PofB20**.

10.7.9 Cliff's delta e p-value

IMPORTANTE PER L'ESAME

Il professore insiste molto sulla differenza tra **p-value** ed **effect size**. Il **p-value** indica quanto sia probabile commettere errore nel rigettare l'ipotesi nulla, ma *non* misura la grandezza dell'effetto. Il **Cliff's delta**, invece, misura proprio quanto l'effetto sia grande e importante.

Nota del Prof

Nel caso della normalizzazione, i valori di **Cliff's delta** risultano per lo più classificati come **Large**, cioè maggiori o uguali a 0.43, mostrando che l'effetto non è solo statisticamente significativo, ma anche sostanzialmente rilevante.

10.8 RQ3

10.8.1 Spearman rank correlation

La RQ3 si chiede se, normalizzando la PofB, il classificatore che prima risultava il migliore resti tale oppure se la graduatoria venga stravolta. Per confrontare i ranking si usa il coefficiente di correlazione di **Spearman**.

10.8.2 Confronto ranking con e senza normalizzazione

Il confronto viene fatto tra le classifiche dei classificatori ottenute con la stessa PofB, ma calcolata **con** e **senza** normalizzazione.

10.8.3 Best classifier

L'analisi si concentra anche su un aspetto pratico fondamentale: verificare se il **miglior classificatore** resti lo stesso prima e dopo la normalizzazione.

10.8.4 Risultati e interpretazione pratica

I risultati mostrano una correlazione solo **modesta** nella maggior parte dei casi. In pratica, identificare il miglior classificatore usando la vecchia PofB equivale spesso a scegliere il classificatore sbagliato rispetto alla più realistica NPofB.

Questo implica che molti studi passati basati sulla PofB non normalizzata risultano fuorvianti, perché potrebbero aver promosso come *miglior modello* un algoritmo che, in realtà, non fornisce il massimo beneficio effort-aware al tester.

10.9 RQ4

10.9.1 Confronto tra diverse NPofBx

Una volta stabilito che la normalizzazione è opportuna, la RQ4 si chiede se basti usare una sola metrica, per esempio **NPofB20**, oppure se il comportamento dei classificatori cambi al variare della soglia x .

10.9.2 Ruolo di x da 10 a 90

L'analisi considera soglie che vanno dal **10%** al **90%**, oltre alla metrica media **Average NPofB**.

10.9.3 Average NPofB e confronto dei ranking

Le diverse classifiche vengono confrontate a coppie tramite **Spearman**, valutando anche il comportamento dell'**Average NPofB**.

10.9.4 Risultati e best classifier

I risultati dimostrano che i ranking dei classificatori sono molto lontani dall'essere identici al variare della soglia x . In circa l'**88%** dei casi, il classificatore migliore cambia se si cambia la quantità di codice da ispezionare.

In altre parole, il modello ottimale per ispezionare il **10%** del codice potrebbe non essere più il migliore se l'obiettivo è ispezionarne il **50%**.

10.10 Figure, tabelle e grafici

Solo nelle slide (non spiegato a voce)

Le tabelle e i grafici descritti qui sono presentati nelle slide come supporto visivo dei risultati, anche quando non vengono spiegati in dettaglio a voce.

10.10.1 Esempio PofB20 e NPofB20

Le tabelle iniziali del paper illustrano visivamente l'esempio dei 7 file, mostrando il passaggio da **33%** a **66%** di bug trovati grazie alla normalizzazione.

10.10.2 Tabella delle EAM usate negli studi precedenti

La **Tabella 4.4** mostra che, su **152** studi primari, **132** usano **0 EAM**. Nessun paper in letteratura ha usato **5 o più EAM**.

10.10.3 Tabella dei relative gains

La **Tabella 4.6** mostra il forte guadagno dovuto alla normalizzazione:

- +414% per **PofB10**;
- guadagni progressivamente decrescenti fino a +11% per **PofB90**;
- tutti i confronti hanno **p-value < 0.0001**.

10.10.4 Tabella degli equivalent best classifiers

La **Tabella 4.8** e il boxplot associato mostrano la proporzione di casi in cui il miglior modello resta lo stesso. Per esempio, si passa dal **15%** per **NPofB10** al **29%** per **NPofB90**. Questo dato è importante perché smonta l'idea che testare una sola EAM sia statisticamente e industrialmente corretto.

10.11 Main results e conclusioni

10.11.1 Superiorità delle NPofBs rispetto alle PofBs

Dal punto di vista statistico e dell'ordine di grandezza, la **NPofB** risulta nettamente superiore alla **PofB**. Fornisce valutazioni più realistiche perché allinea le stime matematiche al comportamento effettivo di uno sviluppatore, che nella pratica considera sempre anche il peso delle dimensioni del file.

10.11.2 Cambiamento del best classifier dopo la normalizzazione

L'impatto più forte della normalizzazione è che essa **ribalta il ranking dei classificatori**. Modelli ritenuti ottimi nella letteratura precedente perdono il primato quando si passa alla valutazione normalizzata.

10.11.3 Implicazioni pratiche

Chiunque valuti un classificatore in ingegneria del software dovrebbe:

- calcolare metriche **Effort-Aware normalizzate**, cioè **NPofB**;
- testare il classificatore su **molteplici soglie x** , e non su una sola;
- evitare conclusioni basate esclusivamente su una singola metrica o su una sola soglia.

10.12 ACUME tool

10.12.1 Scopo

L'assenza di metriche EAM in letteratura è spesso dovuta alla mancanza di librerie standard per calcolarle. Gli autori propongono quindi **ACUME** (**ACcUracY MEtrics**), un tool open-source in Python per automatizzare il processo.

10.12.2 Input e output

Il tool agisce alla fine della pipeline. Prende in input un file **CSV** predetto da strumenti di Machine Learning, come Weka o scikit-learn, con quattro colonne essenziali:

- **ID**
- **Size**
- **Predicted Probability**
- **Actual Defectiveness**

In output produce un singolo file CSV contenente:

- le metriche classiche di accuratezza;
- tutte le EAM discusse nel lavoro.

10.12.3 Utilità per i ricercatori

ACUME evita sprechi di tempo, riduce il rischio di mismatch nei nomi delle metriche, aumenta la generalizzazione dei risultati e migliora la riproducibilità degli esperimenti.

In sintesi

Le metriche classiche, come Precision e Recall, risultano troppo astratte nel mondo reale del testing software. Questo studio mostra che pesare il ranking dividendo la probabilità per la grandezza del file, cioè usando **NPofB**, aumenta drasticamente la capacità di ritrovare i bug veri. Il risultato è un cambiamento profondo nelle modalità con cui l'accademia e i professionisti dovrebbero valutare i classificatori: mai più con una sola soglia e mai più senza normalizzazione.

Capitolo 11

Snoring: rumore nei dataset di defect prediction

11.1 Introduzione e contesto della defect prediction

La **defect prediction** è una tecnica che mira a identificare gli artefatti software, come le classi, che hanno un'alta probabilità di presentare un difetto.

- **Obiettivo:** ridurre i costi e il tempo dedicati alle fasi di testing e code review, permettendo agli sviluppatori di concentrare gli sforzi su specifiche parti critiche del codice.
- **Contesto:** lo studio si posiziona nell'ambito della **within-project defect prediction**, ovvero la classificazione delle classi difettose all'interno di una specifica release di un progetto.
- **Problema dei dati:** l'affidabilità di un modello predittivo dipende interamente dalla qualità del dataset su cui viene addestrato.

Nota del Prof

Questo richiama il principio del **Garbage In, Garbage Out**. Se i dati in ingresso sono inaccurati, i risultati del classificatore saranno inaffidabili.

Solo nelle slide (non spiegato a voce)

Lavori precedenti hanno già identificato altre fonti di rumore nei dataset, come la **misclassification** e l'origine dei difetti, proponendo soluzioni mirate.

11.2 Sleeping defects e snoring classes

- **Sleeping defects (difetti dormienti):** è possibile che un difetto venga scoperto o corretto solo diverse release dopo la sua introduzione, cioè come **Post-Release Defects**. Durante questo intervallo, il difetto “dorme” nel codice.
- **Snoring classes (classi che russano):** è il fenomeno per cui alcune classi sono affette *esclusivamente* da difetti non ancora corretti. Poiché la presenza di un difetto

non può essere nota prima del suo fix, queste classi appaiono erroneamente come pulite. Contenendo difetti dormienti, producono quindi **rumore** nei dataset di defect prediction.

Nota del Prof

La metafora è semplice: il difetto dorme, ma è la classe che “rusa”, creando disturbo. Se una classe ha cinque bug dormienti ma anche un solo bug già identificato, la classe viene comunque etichettata come difettosa e non russa. La classe russa solo quando **tutti** i suoi bug sono dormienti, sfuggendo all’etichettatura.

IMPORTANTE PER L’ESAME

Lo **snoring** introduce nei dataset **solo Falsi Negativi (FN)** e mai Falsi Positivi. Una classe etichettata come difettosa lo è per certo, perché il fix esiste; una classe etichettata come non difettosa potrebbe invece nascondere un difetto dormiente. Per questo motivo, l’ultima release di un software sembrerà sempre perfetta: i bug ci sono, ma non sono ancora stati trovati.

11.3 Come vengono assegnati i difetti alle classi

Per capire da quando una classe è difettosa, si cerca di risalire al momento di introduzione del bug:

1. si considera la prima **Affected Version** riportata nel sistema di tracciamento, ad esempio Jira;
2. se questa informazione manca, si utilizza l’algoritmo **SZZ**.

Nota del Prof

Dal punto di vista pratico, si prendono i ticket chiusi e classificati come difetti, si individuano i commit che li risolvono e si etichettano come difettose le classi toccate da quel commit, andando a ritroso nel tempo fino alla data di iniezione.

11.4 SZZ: funzionamento e limiti

11.4.1 Funzionamento

SZZ è un processo algoritmico per identificare quando un difetto è stato materialmente introdotto nel progetto. Sfrutta il meccanismo di annotazione del sistema di versionamento, come `git blame`, per determinare, partendo dalle righe di codice modificate durante il fix di un bug, quando quelle stesse righe erano state alterate in precedenza.

11.4.2 Limiti

- **Code Additions:** se il fix consiste nell’aggiungere codice mancante, **SZZ** fallisce perché si basa sull’assunzione che il bug sia stato introdotto nelle stesse linee di codice toccate per correggerlo.

Solo nelle slide (non spiegato a voce)

SZZ riesce a trovare solo il **26–29%** dei **bug-introducing changes**, proprio a causa di questa assunzione restrittiva.

11.4.3 Spiegazione del diagramma SZZ nelle slide

Il diagramma mostra un esempio pratico. In **V3**, che rappresenta il fix, il programmatore corregge un ciclo da $i \leq 4$ a $i < 4$ per risolvere il **Bug #123**. Tramite una *diff* con la versione precedente, SZZ traccia all'indietro l'origine di quella riga di codice, identificando **V2** come la modifica che ha introdotto l'errore.

11.5 Obiettivi dello studio e Research Questions

Lo studio si propone di quantificare l'impatto di questo rumore e si articola in cinque **Research Questions (RQ)**:

1. **To what extent do defects sleep?**
Quanto a lungo dormono i difetti?
2. **To what extent do classes snore?**
In che misura le classi russano?
3. **To what extent does snoring impact the evaluation of classifiers accuracy?**
Come impatta la valutazione dei classificatori?
4. **To what extent does snoring impact defect prediction accuracy?**
Come impatta l'accuratezza predittiva?
5. **To what extent is no data better than snoring data?**
Fino a che punto “nessun dato” è meglio di “dati che russano”?

11.6 RQ1: To what extent do defects sleep?

11.6.1 Rationale

L'obiettivo è comprendere quante release trascorrono tra l'iniezione di un difetto e la sua correzione, misurando i difetti dormienti ignoti a chi crea il dataset.

11.6.2 Design

Sono stati selezionati **20 grandi progetti Apache**, tracciati su **Jira** e versionati su **Git**, con almeno **10 release** ed elevato tracciamento tra bug e codice.

11.6.3 Variabili e metriche

- **Variabile indipendente:** nome del bug in Jira.
- **Variabile dipendente:** numero di release dormite, misurate tra iniezione e fix, sia in valore assoluto sia in percentuale rispetto alle release totali.

11.6.4 Risultati

I diagrammi a scatola (**boxplot**) mostrano la distribuzione temporale del fenomeno. Le mediane evidenziano un forte scostamento rispetto allo zero.

11.6.5 Discussione

Il bug medio dorme per ben **7 release**. Nella maggior parte dei progetti, un difetto dorme per più del **19%** delle release esistenti del progetto.

Solo nelle slide (non spiegato a voce)

Esistono casi estremi come il bug **LUCENE-3672**, dormiente per **84 release**, causato da un cambio di librerie esterne.

11.7 RQ2: To what extent do classes snore?

11.7.1 Rationale

Un difetto che dorme non genera automaticamente una classe che russa se un altro difetto nella stessa classe viene scoperto. L'obiettivo è quindi misurare quanto effettivamente le classi siano affette dallo **snoring**.

11.7.2 Design

Si misura come varia la misurazione analizzando un sottoinsieme “sterile”, cioè il primo **5%** del dataset, osservandolo dal futuro. Lo schema nelle slide utilizza l'icona dei **binocoli** per mostrare l'osservatore che si sposta temporalmente in avanti.

11.7.3 Variabili e metriche

- **Variabile indipendente: Progress**, cioè il progresso temporale, pari alla release misurata divisa per il numero totale di release.
- **Variabile dipendente: Missing Rate**, che equivale ai Falsi Negativi, ossia la percentuale di classi difettose che non vengono identificate come tali:

$$\frac{FN}{FN + TP}$$

Questa quantità equivale a **1 - Recall**.

11.7.4 Risultati

Il grafico a linee mostra chiaramente che, all'aumentare del **Progress**, il **Missing Rate** precipita verso lo zero.

11.7.5 Discussione

Per la maggior parte dei progetti, il **Missing Rate** è superiore al **50%** a meno che non si rimuova più del **20%** delle release recenti.

Nota del Prof

Per non avere alcun Missing Rate, cioè per arrivare a **0%**, in media bisogna ignorare l'ultimo **71%** delle release. Questa è la prova che la misurazione richiede molto tempo per stabilizzarsi.

11.8 RQ3: To what extent does snoring impact the evaluation of classifiers accuracy?

11.8.1 Rationale

Ricercatori e praticanti valutano costantemente tecnologie e classificatori. Se lo **snoring** impatta questi test, allora anche le valutazioni riportate in letteratura risultano compromesse da un forte **bias**.

11.8.2 Design

Sono stati impiegati **15 algoritmi di classificazione** differenti. Si è usata la tecnica dell'**holdout** conservando l'ordine cronologico dei dati: **66%** dei dati storici per il training e **33%** per il testing.

11.8.3 Variabili e metriche

- **Variabili dipendenti:** TP, FN, TN, FP.
- **Metriche complesse:** Precision, Recall, F1, Cohen's Kappa, AUC, Matthews.
- **Misura aggiuntiva:** **Relative Bias**, cioè la distanza relativa tra l'accuratezza con rumore e quella senza rumore.
- **Variabile indipendente:** presenza o assenza di snoring nel training e nel testing.

11.8.4 Risultati

I boxplot mostrano un **Relative Bias** altissimo, vicino al **100%** per quasi tutte le metriche. Tra i valori riportati spiccano **Recall = 1.11** e **Kappa = 1.21**, mentre l'**AUC** è molto meno colpita, con valore **0.11**.

11.8.5 Discussione

A causa dello snoring, si rischia di premiare algoritmi sbagliati.

Solo nelle slide (non spiegato a voce)

Valutando su dati affetti da snoring, il classificatore che è realmente il migliore in un progetto viene selezionato correttamente solo nel **45%** dei casi.

11.9 RQ4: To what extent does snoring impact defect prediction accuracy?

11.9.1 Rationale

Stabilito che lo **snoring** è una forma di rumore, si vuole quantificare matematicamente quanto peggiorino i modelli predittivi usati per trovare i bug.

11.9.2 Design

Si testa l'accuratezza di un modello addestrato su dati sporchi, indicati come **TrS** (*Training con Snoring*), contro un test set asettico e privo di rumore, indicato come **TeNS** (*Test No Snoring*).

11.9.3 Variabili e metriche

Le variabili e le metriche considerate sono identiche a quelle della RQ3.

11.9.4 Risultati

Confrontando le distribuzioni nei grafici a scatola, l'accuratezza statistica crolla in modo evidente.

11.9.5 Discussione

Lo **snoring** degrada in maniera drammatica le performance per tutti i **15 classificatori** e su tutte le metriche. La metrica meno impattata in assoluto è la **Precision**.

IMPORTANTE PER L'ESAME

La **Recall** viene demolita mentre la **Precision** subisce danni più lievi perché lo **snoring** nasconde classi difettose etichettandole come pulite, creando moltissimi **Falsi Negativi**. Se addestriamo un modello fornendogli quasi esclusivamente esempi di classi pulite, il modello diventa conservativo e tende a predire quasi sempre “non difettoso”. Predicendo pochissimi bug, mancherà quasi tutti i bersagli veri, facendo crollare la **Recall**; tuttavia, le poche volte in cui lancerà un allarme, tenderà a farlo su basi più solide, mantenendo relativamente stabile la **Precision**. Quando il dataset è pulito, invece, l'aumento dei positivi è reale e la Recall può risalire.

11.10 RQ5: To what extent is no data better than snoring data?

11.10.1 Rationale

Poiché non possiamo rimuovere chirurgicamente i singoli punti rumorosi, perché i bug sono invisibili finché non vengono scoperti, l'unica difesa consiste nell'eliminare intere release recenti, che sappiamo essere ricche di difetti non ancora emersi. La domanda è quindi se rimuovere dati migliori davvero le predizioni.

11.10.2 Design

Si testano i classificatori rimuovendo a scalare le ultime **1, 2, 3 o 4 release (DropCount)** dal set di addestramento. Le classi etichettate esplicitamente come difettose vengono invece preservate, poiché un difetto palese non può essere un falso negativo.

11.10.3 Variabili e metriche

- **Variabile indipendente:** numero di release eliminate.
- **Variabile dipendente:** guadagno relativo sulle metriche di accuratezza.

11.10.4 Risultati

Rimuovere l'ultima release, cioè **+1 DropCount**, garantisce in media un guadagno relativo del **+41%** in **Recall** e del **+33%** in **F1**. Rimuovere **3 o 4** release restituisce invece valori negativi.

11.10.5 Discussione

Ignorare una singola release è sempre meglio che usare tutti i dati. Ignorare **3 o 4** release causa l'effetto inverso e peggiora i modelli.

Nota del Prof

Qui emerge un conflitto ingegneristico tra il principio **Garbage In, Garbage Out**, secondo cui dati freschi ma rumorosi inquinano il modello, e il principio **Size Does Matter**, secondo cui rimuovere troppe release priva il modello di dati recenti utili per l'addestramento. Lo **sweet spot**, cioè il compromesso ideale, è scartare le ultime **1 o 2 release**.

11.11 Conclusioni

- La maggior parte dei difetti dorme per più del **19%** del ciclo di vita del progetto.
- Il **Missing Rate** resta sopra il **50%** finché non si accetta di ignorare l'ultimo **20%** delle release storiche.
- L'errore relativo nella valutazione dei classificatori si aggira intorno al **100%** per quasi tutte le metriche, ad eccezione dell'**AUC**.
- Il rumore dello snoring deprime pesantemente **Recall**, **F1**, **Matthews** e **Cohen's Kappa**.
- Applicare il **Drop** dell'ultima release migliora le predizioni di circa il **30%**.

11.12 Future works

- Sviluppo di metodologie di rimozione del rumore più **fine-grained**, cioè più granulari.
- Combinazione di diverse tecniche di mitigazione del rumore.
- Utilizzo di distribuzioni probabilistiche come input per i modelli predittivi al posto di etichette binarie nette, cioè difettoso/non difettoso.
- Replica dello studio nel contesto della classificazione **Just-In-Time (JIT)**.

INIZIO DEANGELIS

Capitolo 12

Panoramica del modulo di Software Testing

12.1 Introduzione al modulo

Questa è la lezione introduttiva al modulo di **Software Testing** di **Ingegneria del Software 2**. Il modulo ha l'obiettivo di mostrare come migliorare la qualità del software in produzione attraverso processi, metodologie e tecnologie di testing.

Il corso è tenuto da **Guglielmo De Angelis**, ricercatore presso il gruppo **SaKS** del **CNR (IASI)** di Roma. Le sue aree di ricerca includono:

- modellazione dei sistemi software;
- modellazione di proprietà extra-funzionali;
- generazione automatica a partire da modelli;
- architetture orientate ai servizi;
- software testing.

12.2 Informazioni organizzative

Le lezioni si tengono il **mercoledì**, dalle **11:30 alle 13:30**, in **Aula B14**, fino al **3 giugno 2026**.

Nota del Prof

Di comune accordo con l'aula, si faranno due ore di lezione consecutive, iniziando in orario e finendo puntuali, senza pause intermedie. Non sono previsti ponti o riduzioni del calendario.

12.2.1 Canali Teams

Sono previsti due canali principali:

- il **canale Generale**, usato per informazioni di carattere generale e per la condivisione delle slide;

- il **canale Software Testing**, dedicato alle domande specifiche sugli argomenti del modulo e sul progetto d'esame.

12.2.2 Ricevimento e comunicazione

Nota del Prof

È preferibile porre i propri dubbi direttamente in aula, così che tutti possano ascoltare le risposte. In alternativa, è fortemente consigliato usare la mailing list associata al canale Teams, in modo da discutere i problemi collettivamente anche nella lezione successiva.

IMPORTANTE PER L'ESAME

Il docente non legge in modo sincrono i messaggi diretti su Teams. Le richieste vanno fatte sempre via mail, preferibilmente mettendo in copia il canale Teams del modulo. Le mail private al docente devono essere usate solo per questioni strettamente personali.

Quando il corso non è attivo, il ricevimento avviene solo su appuntamento via email.

12.3 Sillabo del modulo

Il sillabo segue una progressione che parte dalle basi e arriva fino alla generazione automatica dei test:

- **Intro and General Concepts;**
- **Test Automation e Continuous Testing;**
- **Unit and Integration Testing;**
- **Approaches to Test Generation;**
- **Adequacy of the Tests: Control Flow Coverage;**
- **Adequacy of the Tests: Mutation Testing;**
- **Coverage-based Approaches to Test Generation.**

12.4 Materiale didattico

Solo nelle slide (non spiegato a voce)

Tra i testi di riferimento compaiono, tra gli altri, **A.P. Mathur** con *Foundations of software testing*, **B. Garcia** e **P. Tahchiev et al.** per la parte su **JUnit**, **W.E. Lewis** per il testing e il miglioramento continuo della qualità, e **J. Humble** con **D. Farley** per la *Continuous Delivery*.

Nota del Prof

Le slide sono sintesi drastiche dei concetti. Studiare solo sulle slide non basta: è necessario approfondire sui libri. Il testo di Mathur è molto utile per la parte teorica, mentre i libri su JUnit sono più vicini a una manualistica tecnica da consultare per la pratica.

12.5 Modalità di esame

Il lavoro richiesto è **strettamente individuale** e consiste nell'applicare le tecniche del corso a **due classi differenti** appartenenti a un **progetto open-source** della **Apache Software Foundation**.

I sorgenti del progetto devono essere disponibili su **GitHub** e lo studente deve configurare l'ambiente di lavoro del progetto.

Nota del Prof

Si consiglia fortemente di partire da un *fork* personale del progetto su GitHub. In questo modo, se durante il lavoro si individua un bug reale, è anche possibile aprire una *pull request* verso il progetto originale.

IMPORTANTE PER L'ESAME

È richiesta la configurazione di un framework di **Continuous Integration**, ad esempio GitHub Actions o TravisCI. È possibile lavorare solo in locale, ma un approccio di questo tipo avrà un impatto negativo sulla valutazione finale.

12.6 Attività richieste: generazione dei test

12.6.1 Scelta delle classi

IMPORTANTE PER L'ESAME

Il docente raccomanda esplicitamente di non scegliere classi banali, né classi legate alla GUI. In questi casi sarebbe difficile dimostrare l'apprendimento dei concetti del corso.

Solo nelle slide (non spiegato a voce)

Se si intuisce che una classe raggiungerà subito il 100% di coverage senza sforzo, è consigliato contattare il docente.

12.6.2 Test manuali

Su ciascuna classe bisogna definire un insieme di test tramite **category partition**, adottando preferibilmente un approccio **black-box** basato sulle funzionalità esplicite o implicite della classe.

12.6.3 Test automatici

Occorre poi generare automaticamente altri test usando tre approcci distinti:

- approcci **randomici**;
- approcci basati su **LLM** e sperimentazione di diversi prompt;
- approcci guidati da **coverage control-flow**.

12.6.4 Integrazione nel build

Nota del Prof

Per evitare problemi di configurazione e interferenze, è consigliabile disabilitare o rimuovere tutti i test nativi del progetto Apache, lasciando nel ciclo di build soltanto i test sviluppati da voi.

12.7 Valutazione della qualità dei test

La qualità dei test deve essere validata scegliendo almeno **due metriche di adeguatezza**, non necessariamente calcolate da tool automatici, e confrontando i valori ottenuti.

Occorre inoltre:

- valutare la significatività dei test rispetto alle funzionalità attese;
- migliorare i test manuali aumentando la loro adeguatezza;
- applicare tecniche di **mutation testing**;
- aggiungere nuovi test per migliorare la robustezza della suite;
- stimare la **reliability** del sistema assumendo un profilo operativo uniforme.

Solo nelle slide (non spiegato a voce)

Per consentire il completamento della build, quando necessario è possibile disabilitare temporaneamente i test che falliscono, purché la cosa venga documentata.

Nota del Prof

Se un test fallisce sulla classe originale, ci sono due possibilità: o il test è stato scritto male e quindi va corretto, oppure avete scoperto un bug reale, che va documentato e gestito separatamente.

12.8 Varianti delle classi e uso degli LLM

La parte finale del progetto riguarda l'evoluzione del sistema e il comportamento dei test quando il codice cambia.

A partire dalla classe originale C_0 , bisogna chiedere a un LLM di generare quattro varianti di refactoring:

$$C_1, C_2, C_3, C_4$$

Nota del Prof

Il fulcro dell'esperimento è fornire all'LLM quantità crescenti di informazione. In particolare:

- C_1 : si fornisce solo la classe originale e si chiede il refactoring;
- C_2 : si chiede il refactoring imponendo il passaggio dei test black-box;
- C_3 : si aggiunge anche il vincolo di passare i test di control flow;
- C_4 : si impone il passaggio di tutti i test, inclusa la mutation analysis.

Occorre poi:

- eseguire tutti i test originari sulle varianti;
- verificare se le funzionalità iniziali siano state mantenute;
- generare nuovi test automatici per le varianti;
- confrontare qualitativamente i test delle varianti con quelli della classe originale.

Nota del Prof

Se la variante C_4 non supera un test che era stato fornito esplicitamente come vincolo all'LLM, significa che l'LLM ha fallito il refactoring.

12.9 Report finale e consegna

Lo studente deve redigere un **report dettagliato** in cui descrive:

- le attività svolte;
- le decisioni prese;
- la metodologia seguita;
- i problemi affrontati;
- i risultati ottenuti.

Il report ideale ha una lunghezza di circa **12 pagine A4**, a interlinea singola, carattere 10, font Arial, in formato PDF.

Nota del Prof

Le 12 pagine si riferiscono al testo descrittivo vero e proprio. Figure, tabelle e listati di codice vanno inseriti alla fine, come allegati, e referenziati nel testo.

IMPORTANTE PER L'ESAME

Non bisogna usare figure, tabelle o didascalie come scorciatoia per spostare fuori dal conteggio l'analisi testuale. Le tabelle mostrano i dati; la discussione e l'interpretazione devono stare nel corpo del report.

12.9.1 File `classes.txt`

Va inoltre consegnato un file di testo chiamato `classes.txt`, contenente i nomi completi delle classi considerate, in ordine alfabetico, nel formato:

$\langle nomePackage \rangle . \langle nomeSubPackage \rangle . \dots \langle nomeClasse \rangle$

12.9.2 Invio e discussione orale

Il report e il file `classes.txt` vanno inviati via email al docente entro le scadenze pubblicate su Teams. L'esame orale consiste nella presentazione e discussione del report.

Nota del Prof

I link al repository GitHub e al framework di CI dovrebbero essere inseriti esplicitamente nel report oppure nella mail di consegna.

12.10 Messaggi chiave della lezione

IMPORTANTE PER L'ESAME

Il docente valuterà la comprensione critica del lavoro svolto. Non basta riportare un valore numerico, per esempio un coverage del 37%: bisogna spiegare perché si è ottenuto quel valore e quali implicazioni ha.

Nota del Prof

Quando si generano le varianti C_1 – C_4 con l'LLM, conviene sempre iniziare una nuova sessione di chat fresca, in modo da evitare che il modello venga influenzato dai prompt precedenti e produca varianti troppo simili tra loro.

In sintesi

Il modulo di Software Testing prevede un esame basato su un progetto individuale rigoroso. Lo studente deve lavorare su due classi non banali di un progetto Apache, generare e valutare test con approcci diversi, integrare i test in una pipeline di build, studiare l'effetto delle mutazioni e delle varianti generate con LLM, e infine documentare tutto in un report tecnico dettagliato da discutere oralmente.